

OOP Interview Questions - SDE Interview Prep

[*] How to Use This Document

Questions are ranked by interview frequency and importance. Top 10 are absolute must-knows, Top 25 covers core OOP concepts, Top 50 adds advanced topics, and Top 100 provides comprehensive coverage for senior roles.

TOP 10 MOST IMPORTANT OOP QUESTIONS

Q1: What are the four main features/pillars of OOPs?

Answer: Encapsulation, Data Abstraction, Inheritance, and Polymorphism.

Polished Answer: The four pillars of Object-Oriented Programming are:

1. **Encapsulation** - Bundling data and methods that operate on that data into a single unit (class) while hiding internal state from outside access through access modifiers like private and public.
2. **Abstraction** - Hiding complex implementation details and exposing only essential features to the user. Achieved through abstract classes and interfaces.
3. **Inheritance** - Allowing a class (child) to acquire properties and methods from another class (parent), promoting code reusability and establishing hierarchical relationships.
4. **Polymorphism** - Enabling the same interface or method to behave differently based on the context. Includes compile-time (method overloading) and runtime (method overriding) polymorphism.

TL;DR: Encapsulation, Abstraction, Inheritance, Polymorphism - the 4 pillars that make code modular, reusable, secure, and maintainable.

Keyword/Key mappings: Encapsulation data hiding + bundling | Abstraction hiding complexity | Inheritance code reusability | Polymorphism many forms

Q2: What is the difference between a Class and an Object?

Answer: A class is a blueprint or template that defines data members and member functions. An object is an instance of a class representing a real-world entity with state, behavior, and identity.

Polished Answer: A **class** is a user-defined data type that serves as a blueprint for creating objects. It defines the properties (attributes) and behaviors (methods) that objects of that type will have. No memory is allocated when a class is defined.

An **object** is an instance of a class - a concrete entity created from the blueprint. It has:

- **State:** Current values of its data members
- **Behavior:** Actions it can perform through its methods
- **Identity:** Unique memory location distinguishing it from other objects

Example: If Car is a class, then BMW, Mercedes, Audi are objects of that class.

TL;DR: Class = blueprint/template; Object = actual instance created from that blueprint with real values.

Keyword/Key mappings: Class blueprint, template, user-defined type | Object instance, real-world entity, state + behavior + identity

Q3: What is the difference between Compile-Time and Runtime Polymorphism?

Answer: Compile-time polymorphism (static/early binding) resolves function calls at compile time using method overloading or operator overloading. Runtime polymorphism (dynamic/late binding) resolves at execution time using method overriding with virtual functions.

Polished Answer:

Aspect	Compile-Time Polymorphism	Runtime Polymorphism
Binding Time	Compile time (early binding)	Runtime (late binding)
Achieved By	Method overloading, operator overloading	Method overriding, virtual functions
Decision Maker	Compiler	JVM/Runtime based on actual object type
Performance	Faster (resolved early)	Slightly slower (dynamic dispatch)
Flexibility	Less flexible	More flexible

Example of Compile-Time:

cpp

```
javascript.snippet JAVASCRIPT
int add(int a, int b) { return a + b; }double add(double a, double b) { return a + b; }
```

Example of Runtime:

cpp

```
javascript.snippet JAVASCRIPT
class Shape {    virtual void draw() = 0;};class Circle : public Shape {    void draw()
override { /* draw circle */ }};
```

TL;DR: Compile-time = overloading (resolved by compiler); Runtime = overriding (resolved by actual object type at execution).

Keyword/Key mappings: Compile-time static, early binding, overloading | Runtime dynamic, late binding, overriding, virtual functions

Q4: What is Encapsulation and how is it implemented?

Answer: Encapsulation is binding data and methods that manipulate them into a single unit (class) while hiding sensitive data from users. Implemented through data hiding (access modifiers) and bundling data with methods.

Polished Answer: Encapsulation is the fundamental OOP principle that combines:

1. **Bundling:** Grouping data members and the methods that operate on them together in a class
2. **Data Hiding:** Restricting direct access to internal state using access specifiers (private, protected)

Implementation:

cpp

```
javascript.snippet JAVASCRIPT

class BankAccount {private:           // Data hiding    double balance;    string
accountNumber
;    public:           // Controlled access    void deposit(double amount) {    if

(amount > 0) balance += amount;    }    double getBalance() {    return balance;

}};
```

Benefits:

- Prevents unauthorized access and accidental modification
- Makes code maintainable - internal changes don't affect external code
- Provides controlled access through getters/setters

TL;DR: Encapsulation = bundling data + methods in a class + hiding data using private/protected with controlled public access.

Keyword/Key mappings: Encapsulation data hiding, bundling, access modifiers, getters/setters

Q5: What is Inheritance? Explain its types and purpose.

Answer: Inheritance allows a class to derive properties and behaviors from another class. Purpose is code reusability and achieving runtime polymorphism. Types: Single, Multiple, Multilevel, Hierarchical, Hybrid.

Polished Answer: Inheritance establishes an "is-a" relationship where a derived class (child) acquires members from a base class (parent).

Purpose:

- Code reusability (write once, use multiple times)
- Extensibility (add new features to existing classes)
- Runtime polymorphism (method overriding)

Types of Inheritance:

Type	Description	Diagram
Single	One child inherits from one parent	Parent Child
Multiple	One child inherits from multiple parents	P1 + P2 Child
Multilevel	Chain of inheritance (ABC)	A B C
Hierarchical	Multiple children from one parent	Parent [Child1, Child2]
Hybrid	Combination of multiple types	Mix of above

Example:

cpp

```

javascript.snippet
JAVASCRIPT

class Animal {          // Base class    void eat() { }};
//
Derived class    void bark() { }};
class Dog : public Animal {

```

TL;DR: Inheritance = "is-a" relationship for code reuse; 5 types (Single, Multiple, Multilevel, Hierarchical, Hybrid).

Keyword/Key mappings: Inheritance is-a, derived class, base class, reusability, extends

Q6: What is Polymorphism? Explain its types with examples.

Answer: Polymorphism means "many forms" - the ability of code, data, or methods to behave differently in different contexts. Types: Compile-time (static) and Runtime (dynamic) polymorphism.

Polished Answer: Polymorphism (poly = many, morph = forms) allows the same interface to perform different actions based on context or object type.

1. Compile-Time Polymorphism (Static):

- Resolved during compilation
- Achieved through **method overloading** and **operator overloading**

Method Overloading Example:

cpp

```
javascript.snippet JAVASCRIPT

class Calculator {
  int add(int a, int b) { return a + b; }
  int add(int a, int b, int c) { return a + b + c; }
  double add(double a, double b) { return a + b; };
};
```

2. Runtime Polymorphism (Dynamic):

- Resolved during execution based on actual object type
- Achieved through **method overriding** with virtual functions

Method Overriding Example:

cpp

```
javascript.snippet JAVASCRIPT

class Animal {public:
  virtual void sound() { cout << "Animal sound"; }};
class Dog : public Animal {public:
  void sound() override { cout << "Bark"; }};
class Cat : public Animal {public:
  void sound() override { cout << "Meow"; }};
// Usage
Animal* a = new Dog();
a->sound(); // Output: Bark (decided at runtime)
```

Benefits: Code flexibility, extensibility, cleaner design.

TL;DR: Polymorphism = one interface, many implementations; Compile-time (overloading) + Runtime (overriding).

Keyword/Key mappings: Polymorphism many forms, overloading, overriding, virtual functions, dynamic dispatch

Q7: What is the difference between Abstraction and Encapsulation?

Answer: Abstraction hides complexity and shows only essential features. Encapsulation hides data and binds it with methods. Abstraction focuses on behavior/design; Encapsulation focuses on data security/implementation.

Polished Answer: These two concepts are often confused but serve different purposes:

Aspect	Abstraction	Encapsulation
Purpose	Hide complexity, show essentials	Hide data, protect internal state
Focus	What the object does (behavior)	How the object protects data
Achieved By	Abstract classes, interfaces	Access modifiers (private, public)
Level	Design-level concept	Implementation-level concept
Example	ATM (user sees withdraw, not internal processing)	BankAccount (balance is private)

Combined Example:

cpp

```

javascript.snippet
JAVASCRIPT

// Abstraction: defining what operations are needed
class Payment {
    virtual void processPayment() {

```

TL;DR: Abstraction = hiding complexity (what it does); Encapsulation = hiding data (how it protects).

Keyword/Key mappings: Abstraction abstract class, interface, essential features | Encapsulation private, data hiding, getters/setters

Q8: What is the difference between Overloading and Overriding?

Answer: Overloading is compile-time polymorphism with multiple methods having the same name but different parameters. Overriding is runtime polymorphism where a subclass provides its own implementation of a parent class method.

Polished Answer:

Aspect	Overloading	Overriding
Polymorphism Type	Compile-time (static)	Runtime (dynamic)
Relationship	Within same class	Between parent and child class
Method Signature	Same name, different parameters	Same name, same parameters
Return Type	Can be different	Must be same or covariant

Binding	Early binding (compile time)	Late binding (runtime)
Purpose	Provide multiple ways to call method	Provide specific implementation

Overloading Example:

cpp

```

javascript.snippet
JAVASCRIPT

void print(int x) { }void print(string s) { }void print(int x, int y) { }

```

Overriding Example:

cpp

```

javascript.snippet
JAVASCRIPT

class Animal {    virtual void makeSound() { cout << "Generic"; }};class Dog : public
Animal
{    void makeSound() override { cout << "Bark"; }};

```

TL;DR: Overloading = same name, different parameters, same class, compile-time.

Overriding = same name, same parameters, parent-child, runtime.

Keyword/Key mappings: Overloading compile-time, same class, different parameters |

Overriding runtime, inheritance, virtual, same signature

Q9: What is a Constructor? Explain its types.

Answer: A constructor is a special member function with the same name as the class, automatically called when an object is created to initialize data members. Types: Default, Parameterized, Copy, and Move constructors.

Polished Answer: A **constructor** is a special method that initializes an object's state when it's created. It has no return type (not even void) and shares the class name.

Types of Constructors:

1. Default Constructor:

cpp

```
javascript.snippet JAVASCRIPT

class Student {public:    Student() { // No parameters        name = "Unknown";    }};
```

2. Parameterized Constructor:

cpp

```
javascript.snippet JAVASCRIPT

class Student {public:    Student(string n) { // Takes parameters        name = n;    }};
```

3. Copy Constructor:

cpp

```
javascript.snippet JAVASCRIPT

class Student {public:    Student(const Student& obj) { // Copies from existing object

    name = obj.name;    }};
```

4. Move Constructor (C++11+):

cpp

```
javascript.snippet JAVASCRIPT

class Student {public:    Student(Student&& obj) noexcept { // Transfers resources

    name = move(obj.name);    }};
```

Key Points:

- Automatically invoked on object creation
- Can be overloaded (multiple constructors)
- If not defined, compiler provides a default one

TL;DR: Constructor = special method for object initialization; Types: Default, Parameterized, Copy, Move; no return type.

Keyword/Key mappings: Constructor initialization, same name as class, no return type | Types default, parameterized, copy, move

Q10: What is the difference between Shallow Copy and Deep Copy?

Answer: Shallow copy copies references/pointers, so both objects share the same memory for dynamic data. Deep copy creates independent copies of all data including dynamically allocated memory.

Polished Answer:

Aspect	Shallow Copy	Deep Copy
Memory	Shares memory for dynamic data	Allocates new memory for each object
Independence	Objects are dependent	Objects are fully independent
Changes	Affect both objects	Don't affect each other
Performance	Faster	Slower (more memory/time)
Safety	Can cause dangling pointers	Safer for complex objects

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Student {public:    string* name; // Dynamically allocated // Shallow
Copy
    (default copy constructor)    Student(const Student& obj) {        name = obj.name;
//
    Both point to same memory!    } // Deep Copy    Student(const Student& obj) {
name = new string(*obj.name); // New memory allocation    };
```

Problem with Shallow Copy: If one object is destroyed, the other has a dangling pointer.

TL;DR: Shallow copy = shared memory (dangerous); Deep copy = independent memory (safe for dynamic data).

Keyword/Key mappings: Shallow copy shared references, same memory | Deep copy independent copy, new memory allocation

TOP 25 OOP QUESTIONS (11-25)

Q11: What are Access Specifiers and their significance in OOPs?

Answer: Access specifiers control visibility and accessibility of class members. Types: public (accessible everywhere), private (only within class), protected (within class and derived classes). They are crucial for achieving encapsulation and data hiding.

Polished Answer: Access specifiers are keywords that determine who can access class members:

Access Specifier	Within Class	Derived Class	Outside Class
public	v	v	v
protected	v	v	
private	v		

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Example {public:    int publicVar;        // Accessible everywhereprotected:    int
protectedVar;    // Accessible in class and derived classesprivate:    int privateVar;

// Only within this class};
```

Significance:

- Enforces encapsulation and data hiding
- Prevents unauthorized data modification
- Provides controlled access through public methods
- Defines clear interfaces for class usage

TL;DR: Access specifiers (public, private, protected) control member visibility and are essential for encapsulation.

Keyword/Key mappings: public accessible everywhere | private class only | protected

Q12: What is an Abstract Class and how is it different from an Interface?

Answer: Abstract class can have both abstract (unimplemented) and concrete methods, instance variables, and constructors. Interface only declares method signatures (contract) that implementing classes must define. A class can extend one abstract class but implement multiple interfaces.

Polished Answer:

Aspect	Abstract Class	Interface
Methods	Abstract + concrete	Only abstract (until Java 8+ default methods)
Variables	Instance variables allowed	Only constants (public static final)
Constructors	Can have constructors	Cannot have constructors
Inheritance	Single (extends one)	Multiple (implements many)
Access Modifiers	Can use all	Methods implicitly public
Purpose	Shared base functionality	Define contract/capability

Abstract Class Example:

cpp

javascript.snippetJAVASCRIPT

```
class Shape {protected:    string color;public:    Shape(string c) : color(c) {} // Constructor    virtual void draw() = 0;        // Pure virtual (abstract)    void setColor(string c)    )    { color = c; } // Concrete method};
```

Interface Example (Java):

java

```
javascript.snippet JAVASCRIPT

interface Drawable {    void draw(); // Implicitly public abstract    int CONSTANT = 10; // Implicitly public static final}
```

When to Use: Use abstract class when classes share common code; use interface when unrelated classes need the same behavior.

TL;DR: Abstract class = partial implementation with shared code; Interface = pure contract with no implementation.

Keyword/Key mappings: Abstract class partial implementation, shared code | Interface contract, capability, multiple inheritance

Q13: What is the Diamond Problem and how is it resolved?

Answer: The Diamond Problem occurs in multiple inheritance when a class inherits the same base class through two different parent classes, causing ambiguity. Resolved in C++ using virtual inheritance; Java avoids it by not supporting multiple class inheritance.

Polished Answer: The Diamond Problem creates ambiguity when:

text

```
javascript.snippet JAVASCRIPT

A (Base Class)    / \    B    C (Both inherit from A)    \ /    D (Inherits from both B and C)
```

Problem: Class D has two copies of class A's members, causing ambiguity.

C++ Solution - Virtual Inheritance:

cpp

```
javascript.snippet JAVASCRIPT

class A {public:    void display() { }};class B : virtual public A { }; // Virtual inheritance
class C : virtual public A { }; // Virtual inheritance
class D : public B, public C { }; //

Now only ONE copy of A
```

Java Solution:

Java disallows multiple class inheritance entirely, eliminating the problem. Multiple inheritance is achieved through interfaces.

Key Points:

- Virtual inheritance ensures only one shared base class instance
- Increases object size slightly (vtable pointers)
- Common interview question for C++ developers

TL;DR: Diamond problem = ambiguous multiple inheritance; Solution = virtual inheritance (C++) or interfaces (Java).

Keyword/Key mappings: Diamond problem multiple inheritance, ambiguity | Virtual inheritance single shared base instance

Q14: What are Virtual Functions and Pure Virtual Functions?

Answer: Virtual functions are methods that can be overridden in derived classes, enabling runtime polymorphism. Pure virtual functions have no implementation (declared with = 0) and make the class abstract; they must be overridden by derived classes.

Polished Answer:

Virtual Function:

- Declared with `virtual` keyword in base class
- Provides default implementation
- Can be overridden in derived classes
- Enables dynamic dispatch based on actual object type

cpp

```
javascript.snippet JAVASCRIPT

class Animal {public:    virtual void sound() {           // Virtual function
cout
    << "Animal makes sound";    }};class Dog : public Animal {public:    void sound()
override
{           // Override        cout << "Dog barks";    }};
```

Pure Virtual Function:

- Declared with = 0
- No implementation in base class
- Makes the class abstract (cannot be instantiated)
- Must be overridden by concrete derived classes

cpp

```
javascript.snippet JAVASCRIPT

class Shape {public:    virtual double area() = 0; // Pure virtual function};class
Circle
: public Shape {    double area() override { return 3.14 * r * r; } // Must implement};
```

Key Points:

- Virtual functions are the mechanism for runtime polymorphism
- Pure virtual functions define interfaces
- Java: all non-static, non-final, non-private methods are virtual
- Python: all methods are virtual by default

TL;DR: Virtual function = overridable with default implementation; Pure virtual = no implementation, must override, makes class abstract.

Keyword/Key mappings: Virtual override, dynamic dispatch, runtime polymorphism | Pure virtual abstract, = 0, interface

Q15: What is the difference between Association, Aggregation, and Composition?

Answer: Association is a general relationship between independent objects. Aggregation is a weak "has-a" relationship where child can exist independently. Composition is a strong "has-a" relationship where child's lifetime depends on parent.

Polished Answer:

Aspect	Association	Aggregation	Composition
Relationship	Uses-a	Weak has-a	Strong has-a
Lifetime	Independent	Child survives parent	Child dies with parent
Ownership	No ownership	Weak ownership	Strong ownership
Coupling	Loose	Moderate	Tight

Examples:

Association (uses-a):

cpp

```
javascript.snippet JAVASCRIPT

class Student { };class Course {    Student* enrolledStudent; // Both can exist independently
};
```

Aggregation (weak has-a):

cpp

```
javascript.snippet JAVASCRIPT

class Employee { };class Company {    vector<Employee*> employees; // Employees survive if company closes};
```

Composition (strong has-a):

cpp

```
javascript.snippet JAVASCRIPT

class Room { };class House {    vector<Room> rooms; // Rooms destroyed with house ~ House
() { /* rooms automatically deleted */ };
```

TL;DR: Association = independent relationship | Aggregation = weak ownership (child independent) | Composition = strong ownership (child dependent).

Keyword/Key mappings: Association uses-a, independent | Aggregation weak has-a, child

Q16: What is a Destructor and can it be overloaded?

Answer: A destructor is a special member function automatically called when an object is destroyed to free resources. It has the class name prefixed with tilde (~). Destructors cannot be overloaded - only one destructor per class.

Polished Answer:

Destructor Characteristics:

- Named with tilde (~) prefix in C++, `__del__` in Python
- No return type, no parameters
- Cannot be overloaded (only one per class)
- Automatically called when object goes out of scope or is deleted
- Used to release resources (memory, files, connections)

Example:

cpp

```
javascript.snippet JAVASCRIPT

class File { FILE* file;public: File() { file = fopen("data.txt", "r"); } ~File() { // Destructor if (file) fclose(file); cout << "File closed"; };
```

Key Points:

- Java doesn't have destructors (uses garbage collector)
- Python uses `__del__` method
- Base class destructor should be virtual when inheritance is used
- C++ automatically calls destructors in reverse order of construction

TL;DR: Destructor = automatically called cleanup method, no overload, one per class.

Keyword/Key mappings: Destructor cleanup, ~classname, automatic, no overloading | Resource cleanup files, memory, connections

Q17: What is the difference between Inheritance and Composition?

Answer: Inheritance represents an "is-a" relationship (Dog is an Animal) with tight coupling. Composition represents a "has-a" relationship (Car has an Engine) with loose coupling. Modern design favors composition for flexibility.

Polished Answer:

Aspect	Inheritance	Composition
Relationship	is-a	has-a
Coupling	Tight	Loose
Flexibility	Less flexible	More flexible
Behavior Acquisition	Automatic (inherited)	Delegated (through objects)
Change Impact	Changes propagate through hierarchy	Changes localized
Design Principle	White-box reuse	Black-box reuse

Inheritance Example:

cpp

```
javascript.snippet JAVASCRIPT
class Animal { };class Dog : public Animal { }; // Dog IS an Animal
```

Composition Example:

cpp

```
javascript.snippet JAVASCRIPT
class Engine { };class Car {   Engine engine; // Car HAS an Engine};
```

Why Favor Composition?

- Easier to change behavior at runtime
- Avoids deep inheritance hierarchies
- Better encapsulation
- Reduces tight coupling between classes

TL;DR: Inheritance = is-a, tight coupling; Composition = has-a, loose coupling, preferred for flexibility.

Keyword/Key mappings: Inheritance is-a, extends, tight coupling | Composition has-a, contains, loose coupling

Q18: What is a Copy Constructor and when is it called?

Answer: A copy constructor is a constructor that initializes an object using another object of the same class. It's called when: passing object by value, returning object by value, and explicitly creating a copy.

Polished Answer:

Syntax:

cpp

```
javascript.snippet JAVASCRIPT

class MyClass {public:    MyClass(const MyClass& obj) { // Copy constructor    // Copy
    data from obj    }};
```

When Called:

1. Passing by value:

cpp

```
javascript.snippet JAVASCRIPT

void func(MyClass obj) { } // Copy constructor called
```

1. Returning by value:

cpp

```
javascript.snippet JAVASCRIPT

MyClass createObject() {    MyClass obj;    return obj; // Copy constructor called}
```

1. Explicit copy:

cpp

```
javascript.snippet JAVASCRIPT

MyClass obj1;MyClass obj2 = obj1; // Copy constructorMyClass obj3(obj1); // Copy constructor
```

Key Points:

- Compiler provides default copy constructor (shallow copy)
- Define custom copy constructor for deep copy with dynamic memory
- Takes const reference to avoid infinite recursion

TL;DR: Copy constructor = creates new object as copy of existing; called on pass/return by value and explicit copy.

Keyword/Key mappings: Copy constructor const reference, object copy, deep/shallow copy

Q19: What are Friend Functions and Friend Classes?

Answer: Friend functions are non-member functions granted access to private and protected members of a class. Friend classes are classes whose member functions can access private/protected members of another class.

Polished Answer:

Friend Function:

cpp

```
javascript.snippet JAVASCRIPT

class Box {private:    int length;public:    Box() : length(10) { }    friend void printLength (Box b); // Friend declaration};void printLength(Box b) {    cout << b.length; // Can access private member!}
```

Friend Class:

cpp

```
javascript.snippet JAVASCRIPT

class A {private:    int secret;public:    friend class B; // Class B is a friend};
class
    B {public:        void accessA(A obj) {            cout << obj.secret; // Can access A's
private
members        }};
```

Key Points:

- Friendship is not mutual (A is friend of B B is friend of A)
- Friendship is not inherited
- Friend functions are not member functions
- Breaks encapsulation (use sparingly)
- Often used for operator overloading

TL;DR: Friend = grants access to private/protected members to non-member functions or other classes.

Keyword/Key mappings: Friend function non-member, private access | Friend class full access to another class

Q20: What is the difference between a Structure and a Class in C++?

Answer: In C++, both are user-defined data types, but structures have public access by default while classes have private access by default. Syntax differs: struct vs class keywords.

Polished Answer:

Aspect	Structure (struct)	Class
Default Access	public	private
Default Inheritance	public	private
Keyword	struct	class
Usage	Simple data containers	Complex objects with behavior

Example:

cpp

```
javascript.snippet JAVASCRIPT

struct Point {    int x;  // Public by default    int y;};class Student {    string
name
;  // Private by default!public:    void setName(string n) { name = n; }};
```

Key Points:

- Both can have constructors, destructors, methods
- Both support inheritance
- Difference is only default access level
- struct commonly used for POD (Plain Old Data)

TL;DR: struct = public by default; class = private by default; otherwise nearly identical in C++.

Keyword/Key mappings: struct public default, data container | class private default, encapsulation

Q21: What is Method Resolution Order (MRO) in Python?

Answer: MRO defines the order in which Python searches for methods in class hierarchies with multiple inheritance. It uses the C3 linearization algorithm for consistent, predictable lookup.

Polished Answer:

MRO: Determines method lookup order when multiple inheritance creates ambiguity.

Example:

python

```
javascript.snippet JAVASCRIPT

class A:    def method(self):        print("A")
class B(A):    def method(self):        print("B")
class C(A):    def method(self):        print("C")
class D(B, C): # Multiple inheritance
    pass
print(D.mro()) # Output: [D, B, C, A, object]
d = D()
d.method() # Output: B (B comes before C in MRO)
```

C3 Linearization Algorithm:

- Ensures consistent ordering

- Preserves local precedence
- Avoids ambiguous hierarchies
- Guarantees each class appears before its parents

Key Points:

- Python searches methods in MRO order
- `super()` follows MRO
- Accessible via `ClassName.mro()` or `ClassName.__mro__`

TL;DR: MRO = method lookup order in multiple inheritance, calculated by C3 algorithm.

Keyword/Key mappings: MRO C3 linearization, method lookup, multiple inheritance | `super()` follows MRO

Q22: Why should a base class destructor be virtual in C++?

Answer: A virtual destructor ensures proper cleanup when deleting a derived object through a base class pointer. Without it, only the base destructor runs, causing memory leaks.

Polished Answer:

Problem Without Virtual Destructor:

cpp

```

class Base {public:    ~Base() { cout << "Base destructor"; }};class Derived : public
Base
{    int* data; // Dynamically allocatedpublic:    Derived() { data = new int[100]; }

~Derived() {        delete[] data; // Never called!        cout <<
"Derived destructor"
;    };Base* ptr = new Derived();delete ptr; // Only Base destructor called  MEMORY
LEAK
!

```

Solution:

cpp

```
class Base {public:    virtual ~Base() { cout << "Base destructor"; } // Virtual destructor};
```

Key Points:

- Rule: If a class has virtual functions, it needs virtual destructor
- Ensures proper cleanup in polymorphic hierarchies
- Slight overhead due to vtable
- Critical for memory management in C++

TL;DR: Virtual destructor = proper cleanup when deleting derived objects through base pointer; prevents memory leaks.

Keyword/Key mappings: Virtual destructor proper cleanup, polymorphic delete, memory leak prevention

Q23: What is the purpose of the `virtual` keyword in C++?

Answer: The `virtual` keyword enables runtime polymorphism by allowing derived classes to override base class methods. It ensures the correct function is called based on the actual object type, not the pointer type.

Polished Answer:**Purpose of `virtual`:**

1. **Method Overriding:** Allows derived classes to provide specific implementations
2. **Dynamic Dispatch:** Correct method called at runtime based on actual object
3. **Polymorphic Behavior:** Same interface, different behaviors

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Shape {public:    virtual void draw() { cout << "Drawing Shape"; }};class Circle
:
    public Shape {public:    void draw() override { cout << "Drawing Circle"; }};class
Rectangle
    : public Shape {public:    void draw() override { cout << "Drawing Rectangle"; }};
Shape
* shapes[] = {new Circle(), new Rectangle()};shapes[0]->draw(); // Drawing Circle (
virtual
dispatch)shapes[1]->draw(); // Drawing Rectangle
```

Without virtual:

cpp

```
javascript.snippet JAVASCRIPT

// Without virtual keywordShape* shape = new Circle();shape->draw(); // Drawing Shape (wrong! stat
```

TL;DR: virtual enables dynamic dispatch; correct method called based on actual object type at runtime.

Keyword/Key mappings: virtual dynamic dispatch, runtime polymorphism, override, vtable

Q24: What are the advantages and disadvantages of OOPs?

Answer: Advantages: code reusability, maintainability, data security, modularity.

Disadvantages: steeper learning curve, complex design, more memory/time overhead.

Polished Answer:

Advantages:

- **Code Reusability:** Inheritance allows reuse of existing code
- **Maintainability:** Modular design makes updates and fixes easier
- **Data Security:** Encapsulation protects data through access modifiers
- **Modularity:** Classes represent separate units for better organization
- **Scalability:** Easier to extend for large applications
- **DRY Principle:** Write once, reuse multiple times
- **Real-world Modeling:** Objects mirror real-world entities

Disadvantages:

- **Learning Curve:** Concepts like inheritance, polymorphism are complex
- **Design Overhead:** Requires careful planning and design
- **Performance:** Objects and virtual functions add overhead
- **Memory Usage:** Objects consume more memory than simple data structures
- **Not Always Necessary:** Overkill for small, simple programs

TL;DR: OOP advantages = reusable, maintainable, secure code; Disadvantages = complex, slower, more memory.

Keyword/Key mappings: Advantages reusability, encapsulation, modularity | Disadvantages complexity, overhead, learning curve

Q25: What is the difference between Procedural Programming and OOP?

Answer: Procedural programming focuses on functions with separate data, less secure, better for small programs. OOP focuses on objects that bundle data and methods, more secure through encapsulation, better for large applications.

Polished Answer:

Aspect	Procedural Programming	OOP
Focus	Functions	Objects
Data & Functions	Separate	Bundled together
Security	Less secure	More secure (encapsulation)
Approach	Top-down	Bottom-up
Reusability	Limited	High (inheritance)
Scalability	Difficult for large apps	Better for large apps
Examples	C, Pascal	Java, C++, Python

Procedural Example:

C

```
javascript.snippet JAVASCRIPT
// Data separate from functions
float radius;
float area(float r) { return 3.14 * r * r; }
```

OOP Example:

cpp

```
javascript.snippet JAVASCRIPT
// Data and methods together
class Circle { float radius; // Data
public: float area() { // M
```

Key Points:

- OOP organizes code around real-world entities
- Procedural organizes code as sequential steps
- OOP provides better abstraction and modularity
- Both have their use cases

TL;DR: Procedural = functions + separate data; OOP = objects bundling data + methods together.

Keyword/Key mappings: Procedural functions, top-down, separate data | OOP objects, bottom-up, bundled data+methods

TOP 50 OOP QUESTIONS (26-50)

Q26: What is the need for OOPs? Why is it preferred?

Answer: OOPs helps understand software easily, increases readability and maintainability, and enables writing and managing very large software efficiently through modular design.

Polished Answer:

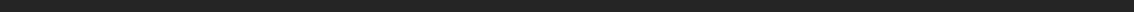
Why OOPs is Needed:

1. **Complexity Management:** Large systems become manageable by breaking them into objects
 2. **Real-world Modeling:** Maps directly to how we perceive the world (entities with properties and behaviors)
 3. **Team Collaboration:** Modular classes allow multiple developers to work simultaneously
-

4. **Maintenance:** Easier to find and fix bugs in isolated classes
5. **Extensibility:** New features can be added without breaking existing code

Example:

cpp



The screenshot shows a code editor window with a title bar containing three colored circles (red, yellow, green) and the text "javascript.snippet". The editor's content area displays the following JavaScript code:

```
// Procedural approach
float calculateArea(float length, float width) {    return length * width;}
float
```

TL;DR: OOPs needed for managing complexity, modeling real-world, improving maintainability and team collaboration.

Keyword/Key mappings: Need complexity management, real-world modeling, maintainability, scalability

Q27: What is meant by Structured Programming?

Answer: Structured Programming is a programming method with organized control flow using blocks containing rules and definitive control flows like if/then/else, while/for loops, block structures, and subroutines.

Polished Answer:

Characteristics:

- Clear control structures (sequence, selection, iteration)
- No goto statements (or minimal use)
- Modular blocks (functions/procedures)
- Top-down design approach
- Single entry, single exit for blocks

Control Structures:

C

```
javascript.snippet JAVASCRIPT

// Sequence
int a = 5; int b = 10; // Selection
if (a > b) {    printf("a is larger"); } else {    printf("b is larger"); }
```

Key Points:

- Nearly all paradigms (including OOP) include structured programming

- Focuses on clear, readable control flow
- Basis for all modern programming

TL;DR: Structured programming = organized control flow with blocks, loops, and conditionals without goto.

Keyword/Key mappings: Structured programming control flow, blocks, loops, conditionals

Q28: What is Garbage Collection in OOPs?

Answer: Garbage collection is the mechanism of automatically managing memory by freeing up memory occupied by objects that are no longer needed, preventing memory-related errors.

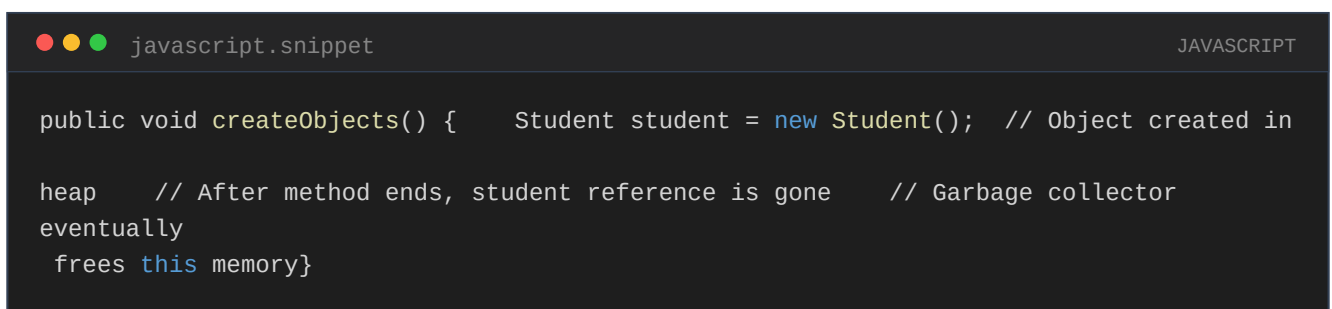
Polished Answer:

Garbage Collection:

- Automatic memory management mechanism
- Identifies and removes objects no longer referenced
- Prevents memory leaks and dangling pointers
- Found in Java, Python, C# (not C++ - manual management)

How It Works (Java Example):

java



```
javascript.snippet                                JAVASCRIPT

public void createObjects() {    Student student = new Student(); // Object created in
                                heap    // After method ends, student reference is gone    // Garbage collector
                                eventually
                                frees this memory}
```

Key Points:

- **C++:** Manual memory management (new/delete, smart pointers)
- **Java:** Automatic GC (mark and sweep, generational)
- **Python:** Reference counting + GC for cycles
- Not deterministic - cannot predict exactly when GC runs

Benefits:

- Prevents memory leaks

- Reduces programmer burden
- Automatic cleanup of unreachable objects

TL;DR: Garbage collection = automatic memory management that frees unused objects; Java/Python have it, C++ uses manual management.

Keyword/Key mappings: Garbage collection automatic memory, unreachable objects, memory leak prevention

Q29: What is exception handling in OOPs?

Answer: Exception handling is the mechanism for identifying undesirable states a program might reach and specifying desirable outcomes, preventing program crashes. Try-catch is the most common method.

Polished Answer:

Exception Handling:

- Detects and handles runtime errors gracefully
- Prevents program termination on unexpected inputs
- Provides alternate execution paths for error conditions

Try-Catch-Finally Structure:

java

```
javascript.snippet JAVASCRIPT

try {    // Code that might throw exception    int result = 10 / 0;    // Throws
ArithmeticException
} catch (ArithmeticException e) {    // Handle specific exception    System.out.println
("
Cannot divide by zero!");} finally {    // Always executed    System.out.println("
Cleanup
code");}
```

Key Components:

- **try:** Contains code that might throw exception
- **catch:** Handles specific exception types
- **finally:** Always executes (cleanup)
- **throw/throws:** Explicitly throw or declare exceptions

Benefits:

- Graceful error handling
- Separates error handling from main logic
- Prevents system crashes
- Provides meaningful error messages

TL;DR: Exception handling = try-catch mechanism to handle runtime errors gracefully without crashing.

Keyword/Key mappings: Exception handling try-catch, runtime errors, graceful recovery | finally always executes

Q30: What is Method Overloading? Provide an example.

Answer: Method overloading is compile-time polymorphism where multiple methods have the same name but different parameter lists (different number, types, or order of parameters).

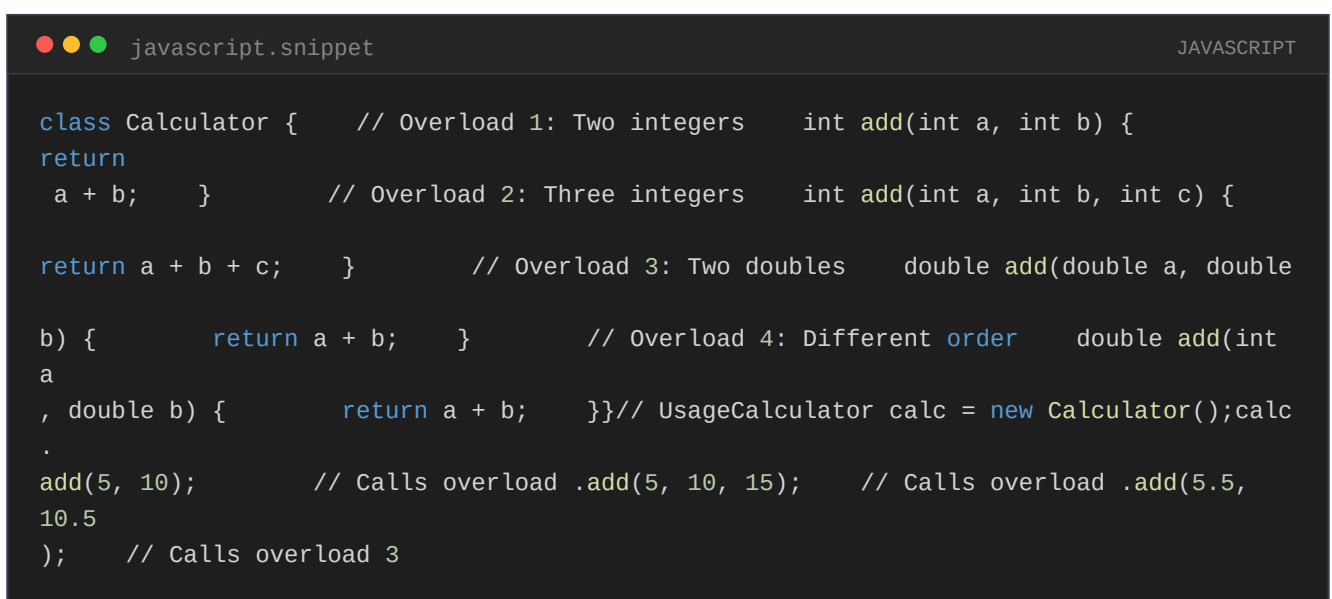
Polished Answer:

Method Overloading Rules:

- Same method name
- Different parameter lists (count, type, or order)
- Return type alone doesn't distinguish methods
- Resolved at compile time

Example:

java



```

class Calculator {
  // Overload 1: Two integers
  int add(int a, int b) {
    return a + b;
  }
  // Overload 2: Three integers
  int add(int a, int b, int c) {
    return a + b + c;
  }
  // Overload 3: Two doubles
  double add(double a, double b) {
    return a + b;
  }
  // Overload 4: Different order
  double add(int a, double b) {
    return a + b;
  }
}

// Usage
Calculator calc = new Calculator();
calc.add(5, 10); // Calls overload 1
calc.add(5, 10, 15); // Calls overload 2
calc.add(5.5, 10.5); // Calls overload 3
calc.add(5, 10.5); // Calls overload 4

```

TL;DR: Overloading = same method name, different parameters, resolved at compile time.

Keyword/Key mappings: Overloading same name, different parameters, compile-time, static polymorphism

Q31: What is Method Overriding? Provide an example.

Answer: Method overriding is runtime polymorphism where a derived class provides its own implementation of a method defined in the parent class with the same signature.

Polished Answer:

Method Overriding Rules:

- Same method name and signature as parent
- Inheritance required (parent-child relationship)
- Parent method must be virtual (C++) or non-final/non-static (Java)
- Return type must be same or covariant

Example:

cpp



```
class Animal {public:    virtual void sound() {        cout << "Animal makes sound";    }};class Dog : public Animal {public:    void sound() override { // Override        cout        << "Dog barks";    }};class Cat : public Animal {public:    void sound() override { //        Override        cout << "Cat meows";    }}; // Usage - Runtime polymorphismAnimal* animal1    = new Dog();Animal* animal2 = new Cat();animal1->sound(); // Dog barksanimal2->sound();    // Cat meows
```

Key Points:

- Uses override keyword (C++11+) for clarity
- Resolved at runtime based on actual object type
- Enables polymorphic behavior

TL;DR: Overriding = same method signature, child provides own implementation, runtime polymorphism.

Keyword/Key mappings: Overriding virtual, override, runtime polymorphism, inheritance, same signature

Q32: What is a Pure Virtual Function?

Answer: A pure virtual function is a virtual function declared with `= 0` in the base class, has no implementation, and must be overridden by derived classes. It makes the class abstract.

Polished Answer:

Characteristics:

- Declared with `= 0`
- No body in base class
- Makes the class abstract (cannot instantiate)
- Derived classes must implement it (unless also abstract)

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Shape {public:    virtual double area() = 0;    // Pure virtual    virtual void
draw
() = 0;    // Pure virtual};class Circle : public Shape {    double radius;public:
double
area() override {        return 3.14    radius    radius;    }    void draw() override {

cout << "Drawing circle";    }}; // Shape s;    // ERROR! Cannot instantiate abstract
classShape
* s = new Circle();    // OK, pointer to abstract class
```

Python Equivalent:

python

```
javascript.snippet JAVASCRIPT

from abc import ABC, abstractmethodclass Shape(ABC):    @abstractmethod    def area(
self
):        pass    # Abstract method
```

TL;DR: Pure virtual function = abstract method with `= 0`, no implementation, makes class abstract.

Keyword/Key mappings: Pure virtual abstract, = 0, interface, must override

Q33: How is Data Abstraction accomplished?

Answer: Data abstraction is accomplished using abstract classes and interfaces, which declare methods without providing implementations, hiding irrelevant details from users.

Polished Answer:

Achieving Abstraction:

1. **Abstract Classes:** Classes with at least one pure virtual function
2. **Interfaces:** Special classes with only method declarations (Java/C#)
3. **Access Modifiers:** Private members hide implementation details
4. **Header Files (C++):** Declaration separated from implementation

Example:

cpp

```
javascript.snippet                                JAVASCRIPT
// Abstract class for abstraction
class Database {public:    virtual void connect() = 0;    // User k
```

Benefits:

- Simplifies user interaction
- Hides implementation complexity
- Allows changing implementation without affecting users

TL;DR: Abstraction achieved through abstract classes/interfaces that hide implementation details from users.

Keyword/Key mappings: Abstraction abstract class, interface, hide implementation, pure virtual

Q34: What is meant by Dynamic Polymorphism?

Answer: Dynamic polymorphism (runtime polymorphism) is when the actual implementation of a function is determined during runtime or execution, achieved through method overriding with virtual functions.

Polished Answer:

Dynamic Polymorphism:

- Function call resolved at runtime
- Based on actual object type, not reference type
- Achieved through virtual functions and overriding
- Uses dynamic dispatch through vtables

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Vehicle {public:    virtual void move() {        cout << "Vehicle moving";    }};
class
    Car : public Vehicle {public:    void move() override {        cout <<
"Car driving on road"
;    }};class Boat : public Vehicle {public:    void move() override {        cout <<
"Boat sailing on water"
;    }};void travel(Vehicle* v) {    v->move(); // Which move() called? Decided at
runtime
!}int main() {    Vehicle* v1 = new Car();    Vehicle* v2 = new Boat();    travel(v1);

// Car driving on road    travel(v2); // Boat sailing on water}
```

Key Points:

- Compiler can't determine which function to call
- Uses vtable for dynamic dispatch
- Slight performance overhead compared to static binding
- Essential for extensible designs

TL;DR: Dynamic polymorphism = runtime resolution of overridden methods through virtual functions.

Keyword/Key mappings: Dynamic polymorphism runtime, overriding, virtual functions, vtable, dynamic dispatch

Q35: What is an Exception? What causes exceptions?

Answer: An exception is a special event raised during program execution at runtime that halts execution. Caused by undesirable situations like invalid input, division by zero, or accessing invalid memory.

Polished Answer:

What is an Exception:

- Runtime error that disrupts normal program flow
- Object containing information about the error
- Can be caught and handled to prevent crash

Common Causes:

1. **Arithmetic Exceptions:** Division by zero
2. **Null Pointer:** Accessing null references
3. **Array Index:** Accessing out-of-bounds index
4. **File I/O:** File not found, permission denied
5. **Network:** Connection timeout
6. **Type Casting:** Invalid type conversion
7. **Out of Memory:** Insufficient heap space

Example:

java

```
javascript.snippet                                JAVASCRIPT

// Causes exception
int[] array = {1, 2, 3};
System.out.println(array[5]); // ArrayIndexOutOfBoundsException
```

TL;DR: Exception = runtime error from invalid operations; caused by bad input, out-of-bounds access, etc.

Keyword/Key mappings: Exception runtime error, invalid operation, program halt | Causes division by zero, null pointer, index out of bounds

Q36: Can static methods be overridden in Java?

Answer: No, static methods cannot be overridden in Java. They belong to the class, not instances, so they cannot participate in runtime polymorphism. What appears as overriding is actually method hiding.

Polished Answer:

Why Static Methods Can't Be Overridden:

- Static methods bind at compile time

- They belong to the class, not instances
- No dynamic dispatch involved
- Parent class reference still calls parent's static method

Example - Method Hiding (NOT Overriding):

java

```

class Parent {    static void display() {        System.out.println("Parent method
");    }}class Child extends Parent {    static void display() { // This is hiding, not overriding
Child method");    }}public class Main {    public static void main(String[] args) {

Parent p = new Child();        p.display(); // Output: Parent method (not Child!)
}}
```

Key Differences:

Aspect	Static Method	Instance Method
Binding	Compile-time	Runtime
Belongs to	Class	Object
Can Override	No (hiding)	Yes

TL;DR: Static methods cannot be overridden; they're hidden. Compile-time binding prevents runtime polymorphism.

Keyword/Key mappings: Static methods class-level, compile-time binding, method hiding | Overriding instance methods only

Q37: How does Java achieve runtime polymorphism?

Answer: Java achieves runtime polymorphism through method overriding and dynamic method dispatch, where a parent class reference points to a child class object and the correct method is called based on the actual object type.

Polished Answer:

Mechanism:

1. **Method Overriding:** Child class provides its own implementation of parent's method
2. **Dynamic Method Dispatch:** JVM determines at runtime which method to call

3. Parent Reference: Parent class reference can point to child object

4. Actual Object Type: Determines which method executes

Example:

java

```
javascript.snippet                                JAVASCRIPT

class Animal {    void sound() {        System.out.println("Animal sound
");    }}class Dog extends Animal {    @Override    void sound() {        System.out.println("
Dog barks
");    }}class Cat extends Animal {    @Override    void sound() {        System.out.println("
Cat meows");    }}public class Main {    public static void main(String[] args) {

Animal a1 = new Dog(); // Parent reference, child object        Animal a2 = new Cat();

a1.sound(); // Dog barks (runtime dispatch)        a2.sound(); // Cat meows    }}
```

Key Points:

- Requires inheritance and method overriding
- Methods must be non-static, non-private, non-final
- JVM uses method tables for dispatch
- Core mechanism for polymorphic behavior

TL;DR: Java runtime polymorphism = method overriding + dynamic method dispatch using parent reference to child object.

Keyword/Key mappings: Runtime polymorphism overriding, dynamic dispatch, parent reference, JVM

Q38: Why does Java not support multiple inheritance using classes?

Answer: Java doesn't support multiple class inheritance to avoid the diamond problem - ambiguity when two parent classes have the same method. Java uses interfaces for multiple inheritance of type.

Polished Answer:

Reason: Java's designers avoided multiple class inheritance to prevent the diamond problem, which creates ambiguity and complexity.

Diamond Problem in Java:

java

```
javascript.snippet JAVASCRIPT

// If Java allowed this: class A { void display() { System.out.println("A"); }} class B extends A
```

Java's Solution - Interfaces:

java

```
javascript.snippet JAVASCRIPT

interface A { void method(); } interface B { default void method() { System.out.println("B's method"); }} interface C { default void method() { System.out.println("C's method"); }} class D implements B, C { // Must explicitly resolve conflict @Override public void method() { B.super.method(); // Explicit choice }}
```

Key Points:

- Classes: Single inheritance only
- Interfaces: Multiple inheritance of type allowed
- Default methods: Explicit resolution required for conflicts
- Keeps language simpler and safer

TL;DR: Java avoids multiple class inheritance to prevent diamond problem; uses interfaces instead.

Keyword/Key mappings: Multiple inheritance not allowed for classes | Interfaces multiple inheritance of type | Diamond problem ambiguity

Q39: What are SOLID principles in OOPs?

Answer: SOLID is a set of five design principles: Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. They improve maintainability, extensibility, and testability.

Polished Answer:

Principle	Description	Benefit
S - Single Responsibility	Class has one reason to change	Easier testing, less coupling
O - Open-Closed	Open for extension, closed for modification	Add features without breaking existing code
L - Liskov Substitution	Subclasses substitutable for parents	Correct polymorphic behavior
I - Interface Segregation	Small focused interfaces over monolithic ones	Clients only depend on what they use
D - Dependency Inversion	Depend on abstractions, not concretions	Decoupled architecture

Examples:

SRP:

java

```

javascript.snippet                                JAVASCRIPT

// Bad - Multiple responsibilities
class User {    void saveUser() { }    void sendEmail() { }    void ...

```

OCP:

java

```

javascript.snippet                                JAVASCRIPT

// Open for extension via interfaces
interface Shape { double area(); }
class Circle implements Shape {
    ...
}

```

TL;DR: SOLID = 5 principles for better OOP design; SRP, OCP, LSP, ISP, DIP.

Keyword/Key mappings: SOLID design principles, maintainability | SRP one responsibility | OCP extensible | LSP substitutable | ISP focused interfaces | DIP depend on abstractions

Q40: What is the Liskov Substitution Principle (LSP)?

Answer: LSP states that a subclass should be fully substitutable for its parent class without breaking the application. Swapping a subclass for its parent should not cause unexpected behavior.

Polished Answer:

Liskov Substitution Principle:

- Named after Barbara Liskov
- Subtypes must be replaceable by their base types
- A program using a base class should work correctly with any derived class
- "Is-a" relationship must be semantically correct

Violating LSP:

java

```
javascript.snippet JAVASCRIPT

class Rectangle {    protected int width, height;        void setWidth(int w) { width =
w; }    void setHeight(int h) { height = h; }    int area() { return width * height; }}
class
Square extends Rectangle {    @Override    void setWidth(int w) {        width = w;

height = w; // Forces square behavior    }    @Override    void setHeight(int h) {

height = h;        width = h; // Violates expectations!    }}// Code that breaks:void
resizeRectangle
(Rectangle r) {    r.setWidth(5);    r.setHeight(10);    // For Square, area = 100 (
10*10
), but expected 50 (5*10)}
```

Correct Design:

java

```
javascript.snippet JAVASCRIPT

interface Shape {    int area();}class Rectangle implements Shape {    int width,
height
;    public int area() { return width * height; }}class Square implements Shape {    int
side;    public int area() { return side * side; }}
```

TL;DR: LSP = subclasses must be substitutable for parents without breaking behavior or expectations.

Keyword/Key mappings: LSP substitutability, inheritance correctness, semantic is-a, behavioral subtyping

Q41: What is the Single Responsibility Principle (SRP)?

Answer: SRP states that a class should have only one reason to change, meaning it should have only one job or responsibility. This improves maintainability and testability.

Polished Answer:

Single Responsibility Principle:

- Each class has exactly one responsibility
- One reason to change = one responsibility
- Reduces coupling and increases cohesion
- Makes classes easier to understand, test, and maintain

Violating SRP:

java

```
javascript.snippet JAVASCRIPT

// Bad - This class has multiple responsibilities
class Employee {    void calculateSalary() { }
```

Correct SRP:

java

```
javascript.snippet JAVASCRIPT

class Employee {    String name;    String department;    // Employee data only}
class SalaryCalculator
{    void calculateSalary(Employee e) { }}
class EmployeeRepository {    void save(
Employee
e) { }}
class EmailService {    void sendEmail(Employee e) { }}
class ReportGenerator {
void generateReport(Employee e) { }}
```

Benefits:

- Easier to test each class independently
- Changes isolated to one class
- Better code organization
- Improved readability

TL;DR: SRP = one class, one responsibility, one reason to change.

Keyword/Key mappings: SRP single responsibility, one reason to change, separation of concerns, cohesion

Q42: What is the Open-Closed Principle (OCP)?

Answer: OCP states that software entities should be open for extension but closed for modification. New functionality should be added by extending existing code, not modifying it.

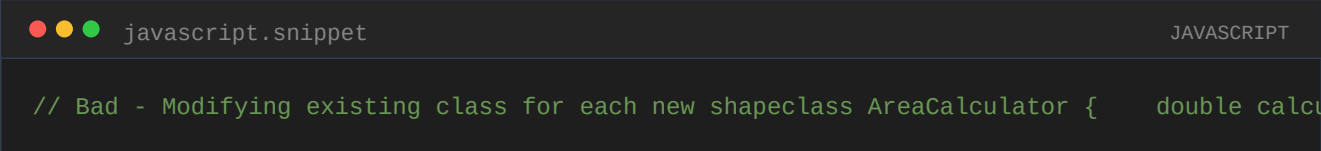
Polished Answer:

Open-Closed Principle:

- **Open for Extension:** Can add new behavior
- **Closed for Modification:** Existing tested code unchanged
- Achieved through abstraction, interfaces, inheritance

Violating OCP:

java

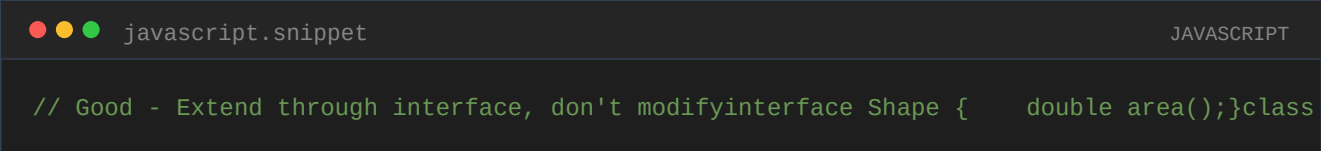


```
javascript.snippet JAVASCRIPT

// Bad - Modifying existing class for each new shapeclass AreaCalculator {    double calculateArea()
```

Correct OCP:

java



```
javascript.snippet JAVASCRIPT

// Good - Extend through interface, don't modifyinterface Shape {    double area();}class Circle im
```

TL;DR: OCP = open for extension (add new classes), closed for modification (don't change existing code).

Keyword/Key mappings: OCP extension vs modification, interfaces, abstraction, extensibility

Q43: What is the Dependency Inversion Principle (DIP)?

Answer: DIP states that high-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.

Polished Answer:

Dependency Inversion Principle:

- Decouples high-level business logic from low-level implementations
- Both depend on interfaces/abstract classes
- Enables swapping implementations easily
- Improves testability (mock objects)

Violating DIP:

java

```
javascript.snippet JAVASCRIPT
// Bad - High-level depends on low-level directly
class MySQLDatabase {    void save(String data) {
```

Correct DIP:

java

```
javascript.snippet JAVASCRIPT
// Good - Both depend on abstraction
interface Database {    void save(String data);}
class MySQLData
```

TL;DR: DIP = depend on abstractions, not concrete implementations; enables flexibility and testability.

Keyword/Key mappings: DIP abstractions, dependency injection, decoupling, interfaces

Q44: What is the Interface Segregation Principle (ISP)?

Answer: ISP states that clients should not be forced to depend on methods they do not use. Prefer small, focused interfaces over large, monolithic ones.

Polished Answer:

Interface Segregation Principle:

- Avoid "fat" interfaces with unrelated methods
- Split into smaller, specific interfaces
- Clients only implement what they need
- Reduces coupling and unnecessary dependencies

Violating ISP:

java

```
javascript.snippet JAVASCRIPT
// Bad - Fat interface forcing unwanted methods
interface Worker {    void work();    void eat();
```

Correct ISP:

java

```
javascript.snippet JAVASCRIPT
// Good - Focused interfaces
interface Workable {    void work();}
interface Eatable {    void eat();}
```

TL;DR: ISP = small focused interfaces; don't force clients to implement methods they don't need.

Keyword/Key mappings: ISP focused interfaces, avoid fat interfaces, client-specific interfaces

Q45: What is meant by static polymorphism?

Answer: Static polymorphism (compile-time polymorphism) is when an object is linked with its function or operator based on values during compile time. Achieved through method overloading and operator overloading.

Polished Answer:

Static Polymorphism:

- Resolved at compile time
- Also known as early binding
- Compiler decides which function to call
- Based on method signature (name + parameters)

Types:

1. Function/Method Overloading:

cpp

```
javascript.snippet JAVASCRIPT

void print(int x) { cout << "Integer: " << x; }void print(string s) { cout <<
"String: "
<< s; }void print(double d) { cout << "Double: " << d; }
```

1. Operator Overloading:

cpp

```
javascript.snippet JAVASCRIPT

class Complex {public:    int real, imag;    Complex operator+(const Complex& other) {
Complex result;        result.real = real + other.real;        result.imag = imag +
other
.imag;        return result;    }};
```

1. Templates (C++):

cpp

```
javascript.snippet JAVASCRIPT

template<typename T>T add(T a, T b) { return a + b; }
```

TL;DR: Static polymorphism = compile-time resolution through overloading and templates.

Keyword/Key mappings: Static polymorphism compile-time, early binding, overloading, templates

Q46: What is the difference between static polymorphism and dynamic polymorphism?

Answer: Static polymorphism resolves function calls at compile time using overloading.

Dynamic polymorphism resolves at runtime using overriding and virtual functions based on actual object type.

Polished Answer:

Aspect	Static Polymorphism	Dynamic Polymorphism
Binding	Compile-time (early)	Runtime (late)
Achieved By	Overloading, templates	Overriding, virtual functions
Decision Maker	Compiler	JVM/Runtime
Performance	Faster	Slightly slower (vtable lookup)
Flexibility	Less flexible	More flexible
Relationship	Same class	Parent-child classes
Method Signature	Different parameters	Same parameters

Static Example:

cpp

```

class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
};

```

Dynamic Example:

cpp

```

class Animal {public:
    virtual void sound() { };
};
class Dog : public Animal {
    void sound() override { cout << "Bark"; };
};

```

TL;DR: Static = compile-time, overloading, same class; Dynamic = runtime, overriding, inheritance.

Keyword/Key mappings: Static compile-time, overloading | Dynamic runtime, overriding, virtual

Q47: How does C++ support Polymorphism?

Answer: C++ supports compile-time polymorphism through templates, function overloading, and default arguments. Runtime polymorphism is supported through virtual functions and

function overriding.

Polished Answer:

1. Compile-Time Polymorphism in C++:

Function Overloading:

cpp

```
javascript.snippet JAVASCRIPT

int add(int a, int b) { return a + b; }double add(double a, double b) { return a + b; }
```

Operator Overloading:

cpp

```
javascript.snippet JAVASCRIPT

class Vector {public:    Vector operator+(const Vector& v) { }};
```

Templates:

cpp

```
javascript.snippet JAVASCRIPT

template<typename T>T max(T a, T b) {    return (a > b) ? a : b;}
```

Default Arguments:

cpp

```
javascript.snippet JAVASCRIPT

int multiply(int a, int b = 2) { return a * b; }// Can call multiply(5) or multiply(5,
3
)
```

2. Runtime Polymorphism in C++:

cpp

```
javascript.snippet JAVASCRIPT

class Base {public:    virtual void print() {        cout << "Base print";    }};class
Derived
: public Base {public:    void print() override {        cout << "Derived print";
}};
Base* b = new Derived();b->print(); // Derived print (runtime dispatch)
```

TL;DR: C++ supports both compile-time (templates, overloading) and runtime (virtual functions) polymorphism.

Keyword/Key mappings: C++ polymorphism templates, overloading, virtual functions, override

Q48: What is a subclass and superclass?

Answer: A subclass (child class) is a class that inherits from another class. A superclass (parent/base class) is the class being inherited from. The subclass acquires members of the superclass.

Polished Answer:

Superclass (Parent/Base Class):

- The class being inherited from
- Provides common properties and methods
- Can be extended by multiple subclasses

Subclass (Child/Derived Class):

- The class that inherits from superclass
- Acquires superclass members
- Can add its own members
- Can override inherited methods

Example:

java

```
javascript.snippet                                JAVASCRIPT

class Vehicle {                                // Superclass    String brand;    void start() {                                System
.
out.println("Vehicle started
");    }}class Car extends Vehicle { // Subclass    int wheels = 4;    @Override    void start
Car started
");    }    void honk() { // New method in subclass    System.out.println("
Beep beep!");    }}
```

Key Points:

- Subclass inherits all public and protected members
- Private members are not directly accessible
- Subclass can extend or modify superclass behavior
- Inheritance establishes hierarchy

TL;DR: Superclass = parent class being inherited; Subclass = child class that inherits from parent.

Keyword/Key mappings: Superclass parent, base class | Subclass child, derived class, extends

Q49: What is Method Hiding in Java?

Answer: Method hiding occurs when a subclass defines a static method with the same signature as a static method in the parent class. The parent's method is hidden, not overridden. Method resolution happens at compile time.

Polished Answer:

Method Hiding vs Overriding:

Aspect	Method Hiding	Method Overriding
Method Type	Static methods	Instance methods
Binding	Compile-time	Runtime
Resolution	Based on reference type	Based on object type
Keyword	No special keyword needed	@Override

Method Hiding Example:

```

class Parent {    static void display() {        System.out.println("Parent static
method
");    }}class Child extends Parent {    static void display() { // Hides parent's method
Child static method");    }}public class Main {    public static void main(String[]
args
) {        Parent p = new Child();        p.display(); // Output: Parent static method

(compile-time binding)        Child c = new Child();        c.display(); //
Output
: Child static method    }}

```

Key Points:

- Static methods cannot be overridden, only hidden
- Method resolution based on reference type, not object type
- No dynamic dispatch for static methods
- Often confused with overriding

TL;DR: Method hiding = static method with same signature in child; resolved at compile time, not runtime.

Keyword/Key mappings: Method hiding static methods, compile-time binding, reference type

Q50: What are some other programming paradigms besides OOPs?

Answer: Other paradigms include Procedural, Functional, Logical, Parallel, and Database programming. They're classified under Imperative (how to execute) and Declarative (what to execute) paradigms.

Polished Answer:**Programming Paradigms Classification:****1. Imperative Paradigms (How to execute):**

- **Procedural:** Sequential steps (C, Pascal)
 - **Object-Oriented:** Objects with data and behavior (Java, C++)
 - **Parallel:** Concurrent execution (CUDA, OpenMP)
-

2. Declarative Paradigms (What to execute):

- **Functional:** Pure functions, immutability (Haskell, Scala)
- **Logical:** Formal logic rules (Prolog)
- **Database:** Query-based (SQL)

Examples:

Procedural:

C

```
javascript.snippet JAVASCRIPT
int sum = 0;for (int i = 0; i < 10; i++) {    sum += i;}
```

Functional:

haskell

```
javascript.snippet JAVASCRIPT
-- No loops, use recursionsum [1..10]
```

Logical:

prolog

```
javascript.snippet JAVASCRIPT
parent(john, mary).parent(mary, alice).grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

Key Points:

- OOP is one of many paradigms
- Languages can support multiple paradigms
- Python supports OOP, functional, and procedural
- Choice depends on problem requirements

TL;DR: Paradigms = Imperative (Procedural, OOP, Parallel) and Declarative (Functional, Logical, Database).

Keyword/Key mappings: Paradigms imperative, declarative | Imperative procedural, OOP, parallel | Declarative functional, logical, database

TOP 100 OOP QUESTIONS (51-100)

Q51: What is the need for OOPs? Why is it preferred?

Answer: OOPs helps users understand software easily, increases readability and maintainability, and enables writing and managing very large software efficiently through modular design.

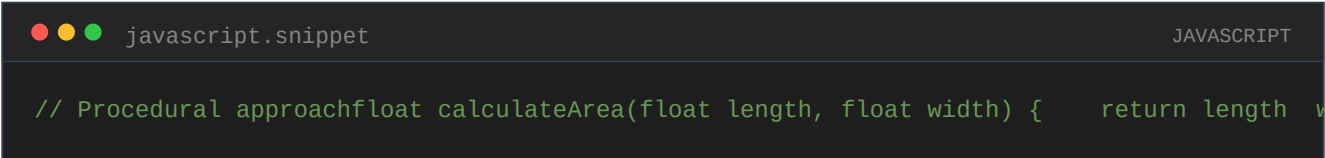
Polished Answer:

Why OOPs is Needed:

1. **Complexity Management:** Large systems become manageable by breaking them into objects
2. **Real-world Modeling:** Maps directly to how we perceive the world (entities with properties and behaviors)
3. **Team Collaboration:** Modular classes allow multiple developers to work simultaneously
4. **Maintenance:** Easier to find and fix bugs in isolated classes
5. **Extensibility:** New features can be added without breaking existing code

Example:

cpp



```
javascript.snippet JAVASCRIPT
// Procedural approachfloat calculateArea(float length, float width) { return length width;}flo
```

TL;DR: OOPs needed for managing complexity, modeling real-world, improving maintainability and team collaboration.

Keyword/Key mappings: Need complexity management, real-world modeling, maintainability, scalability

Q52: What is meant by Structured Programming?

Answer: Structured Programming is a programming method with organized control flow using blocks containing rules and definitive control flows like if/then/else, while/for loops, block structures, and subroutines.

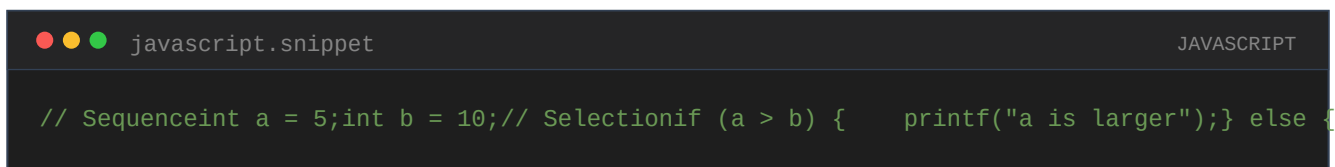
Polished Answer:

Characteristics:

- Clear control structures (sequence, selection, iteration)
- No goto statements (or minimal use)
- Modular blocks (functions/procedures)
- Top-down design approach
- Single entry, single exit for blocks

Control Structures:

C

A screenshot of a code editor window titled 'javascript.snippet' with 'JAVASCRIPT' in the top right corner. The code inside is:

```
// Sequence
int a = 5; int b = 10;
// Selection
if (a > b) {    printf("a is larger"); } else {    printf("b is larger"); }
```

Key Points:

- Nearly all paradigms (including OOP) include structured programming
- Focuses on clear, readable control flow
- Basis for all modern programming

TL;DR: Structured programming = organized control flow with blocks, loops, and conditionals without goto.

Keyword/Key mappings: Structured programming control flow, blocks, loops, conditionals

Q53: What is Garbage Collection in OOPs?

Answer: Garbage collection is the mechanism of automatically managing memory by freeing up memory occupied by objects that are no longer needed, preventing memory-related errors.

Polished Answer:

Garbage Collection:

- Automatic memory management mechanism
- Identifies and removes objects no longer referenced
- Prevents memory leaks and dangling pointers
- Found in Java, Python, C# (not C++ - manual management)

How It Works (Java Example):

java

```
javascript.snippet JAVASCRIPT

public void createObjects() {    Student student = new Student(); // Object created in
heap    // After method ends, student reference is gone    // Garbage collector
eventually
    frees this memory}
```

Key Points:

- **C++:** Manual memory management (new/delete, smart pointers)
- **Java:** Automatic GC (mark and sweep, generational)
- **Python:** Reference counting + GC for cycles
- Not deterministic - cannot predict exactly when GC runs

Benefits:

- Prevents memory leaks
- Reduces programmer burden
- Automatic cleanup of unreachable objects

TL;DR: Garbage collection = automatic memory management that frees unused objects; Java/Python have it, C++ uses manual management.

Keyword/Key mappings: Garbage collection automatic memory, unreachable objects, memory leak prevention

Q54: What is exception handling in OOPs?

Answer: Exception handling is the mechanism for identifying undesirable states a program might reach and specifying desirable outcomes, preventing program crashes. Try-catch is the most common method.

Polished Answer:

Exception Handling:

- Detects and handles runtime errors gracefully
- Prevents program termination on unexpected inputs
- Provides alternate execution paths for error conditions

Try-Catch-Finally Structure:

java

```
javascript.snippet JAVASCRIPT

try {    // Code that might throw exception    int result = 10 / 0;    // Throws
ArithmeticException
} catch (ArithmeticException e) {    // Handle specific exception    System.out.println
("
Cannot divide by zero!");} finally {    // Always executed    System.out.println("
Cleanup
code");}
```

Key Components:

- **try:** Contains code that might throw exception
- **catch:** Handles specific exception types
- **finally:** Always executes (cleanup)
- **throw/throws:** Explicitly throw or declare exceptions

Benefits:

- Graceful error handling
- Separates error handling from main logic
- Prevents system crashes
- Provides meaningful error messages

TL;DR: Exception handling = try-catch mechanism to handle runtime errors gracefully without crashing.

Keyword/Key mappings: Exception handling try-catch, runtime errors, graceful recovery | finally always executes

Q55: What is Method Overloading? Provide an example.

Answer: Method overloading is compile-time polymorphism where multiple methods have the same name but different parameter lists (different number, types, or order of parameters).

Polished Answer:

Method Overloading Rules:

- Same method name
- Different parameter lists (count, type, or order)

- Return type alone doesn't distinguish methods
- Resolved at compile time

Example:

java

```

class Calculator {    // Overload 1: Two integers    int add(int a, int b) {
return
a + b;    }    // Overload 2: Three integers    int add(int a, int b, int c) {
return a + b + c;    }    // Overload 3: Two doubles    double add(double a, double
b) {    return a + b;    }    // Overload 4: Different order    double add(int
a
, double b) {    return a + b;    }}// UsageCalculator calc = new Calculator();calc
.add(5, 10);    // Calls overload .add(5, 10, 15);    // Calls overload .add(5.5,
10.5
);    // Calls overload 3

```

TL;DR: Overloading = same method name, different parameters, resolved at compile time.

Keyword/Key mappings: Overloading same name, different parameters, compile-time, static polymorphism

Q56: What is Method Overriding? Provide an example.

Answer: Method overriding is runtime polymorphism where a derived class provides its own implementation of a method defined in the parent class with the same signature.

Polished Answer:

Method Overriding Rules:

- Same method name and signature as parent
- Inheritance required (parent-child relationship)
- Parent method must be virtual (C++) or non-final/non-static (Java)
- Return type must be same or covariant

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Animal {public:    virtual void sound() {        cout << "Animal makes sound";

}};class Dog : public Animal {public:    void sound() override { // Override
cout
    << "Dog barks";    };};class Cat : public Animal {public:    void sound() override {
//
    Override        cout << "Cat meows";    };};// Usage - Runtime polymorphismAnimal*
animal1
    = new Dog();Animal* animal2 = new Cat();animal1->sound(); // Dog barksanimal2->sound
();
    // Cat meows
```

Key Points:

- Uses `override` keyword (C++11+) for clarity
- Resolved at runtime based on actual object type
- Enables polymorphic behavior

TL;DR: Overriding = same method signature, child provides own implementation, runtime polymorphism.

Keyword/Key mappings: Overriding `virtual`, `override`, runtime polymorphism, inheritance, same signature

Q57: What is a Pure Virtual Function?

Answer: A pure virtual function is a virtual function declared with `= 0` in the base class, has no implementation, and must be overridden by derived classes. It makes the class abstract.

Polished Answer:

Characteristics:

- Declared with `= 0`
- No body in base class
- Makes the class abstract (cannot instantiate)
- Derived classes must implement it (unless also abstract)

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Shape {public:    virtual double area() = 0;    // Pure virtual    virtual void
draw
() = 0;    // Pure virtual};class Circle : public Shape {    double radius;public:
double
area() override {        return 3.14    radius    radius;    }    void draw() override {

cout << "Drawing circle";    }}; // Shape s;    // ERROR! Cannot instantiate abstract
classShape
* s = new Circle();    // OK, pointer to abstract class
```

Python Equivalent:

python

```
javascript.snippet JAVASCRIPT

from abc import ABC, abstractmethodclass Shape(ABC):    @abstractmethod    def area(
self
):        pass    # Abstract method
```

TL;DR: Pure virtual function = abstract method with = 0, no implementation, makes class abstract.

Keyword/Key mappings: Pure virtual abstract, = 0, interface, must override

Q58: How is Data Abstraction accomplished?

Answer: Data abstraction is accomplished using abstract classes and interfaces, which declare methods without providing implementations, hiding irrelevant details from users.

Polished Answer:

Achieving Abstraction:

1. **Abstract Classes:** Classes with at least one pure virtual function
2. **Interfaces:** Special classes with only method declarations (Java/C#)
3. **Access Modifiers:** Private members hide implementation details
4. **Header Files (C++):** Declaration separated from implementation

Example:

cpp

```
javascript.snippet JAVASCRIPT
// Abstract class for abstractionclass Database {public:    virtual void connect() = 0; // User k
```

Benefits:

- Simplifies user interaction
- Hides implementation complexity
- Allows changing implementation without affecting users

TL;DR: Abstraction achieved through abstract classes/interfaces that hide implementation details from users.

Keyword/Key mappings: Abstraction abstract class, interface, hide implementation, pure virtual

Q59: What is meant by Dynamic Polymorphism?

Answer: Dynamic polymorphism (runtime polymorphism) is when the actual implementation of a function is determined during runtime or execution, achieved through method overriding with virtual functions.

Polished Answer:

Dynamic Polymorphism:

- Function call resolved at runtime
- Based on actual object type, not reference type
- Achieved through virtual functions and overriding
- Uses dynamic dispatch through vtables

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Vehicle {public:    virtual void move() {        cout << "Vehicle moving";    }};
class
    Car : public Vehicle {public:    void move() override {        cout <<
"Car driving on road"
;    }};class Boat : public Vehicle {public:    void move() override {        cout <<
"Boat sailing on water"
;    }};void travel(Vehicle* v) {    v->move(); // Which move() called? Decided at
runtime
!}int main() {    Vehicle* v1 = new Car();    Vehicle* v2 = new Boat();    travel(v1);

// Car driving on road    travel(v2); // Boat sailing on water}
```

Key Points:

- Compiler can't determine which function to call
- Uses vtable for dynamic dispatch
- Slight performance overhead compared to static binding
- Essential for extensible designs

TL;DR: Dynamic polymorphism = runtime resolution of overridden methods through virtual functions.

Keyword/Key mappings: Dynamic polymorphism runtime, overriding, virtual functions, vtable, dynamic dispatch

Q60: What is an Exception? What causes exceptions?

Answer: An exception is a special event raised during program execution at runtime that halts execution. Caused by undesirable situations like invalid input, division by zero, or accessing invalid memory.

Polished Answer:

What is an Exception:

- Runtime error that disrupts normal program flow
- Object containing information about the error
- Can be caught and handled to prevent crash

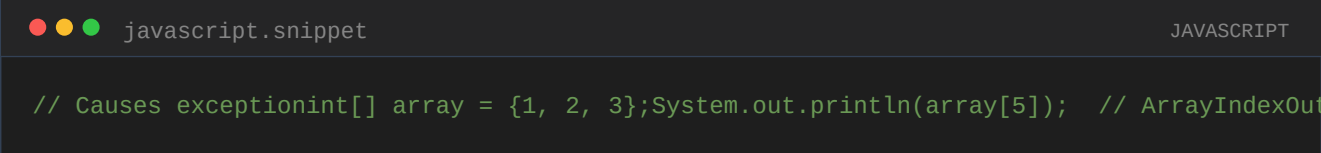
Common Causes:

1. **Arithmetic Exceptions:** Division by zero
2. **Null Pointer:** Accessing null references

3. **Array Index:** Accessing out-of-bounds index
4. **File I/O:** File not found, permission denied
5. **Network:** Connection timeout
6. **Type Casting:** Invalid type conversion
7. **Out of Memory:** Insufficient heap space

Example:

java

A screenshot of a code editor window titled 'javascript.snippet' with 'JAVASCRIPT' in the top right corner. The code inside is:

```
// Causes exceptionint[] array = {1, 2, 3};System.out.println(array[5]); // ArrayIndexOutOfBoundsException
```

TL;DR: Exception = runtime error from invalid operations; caused by bad input, out-of-bounds access, etc.

Keyword/Key mappings: Exception runtime error, invalid operation, program halt | Causes division by zero, null pointer, index out of bounds

Q61: Can static methods be overridden in Java?

Answer: No, static methods cannot be overridden in Java. They belong to the class, not instances, so they cannot participate in runtime polymorphism. What appears as overriding is actually method hiding.

Polished Answer:

Why Static Methods Can't Be Overridden:

- Static methods bind at compile time
- They belong to the class, not instances
- No dynamic dispatch involved
- Parent class reference still calls parent's static method

Example - Method Hiding (NOT Overriding):

java

```
javascript.snippet JAVASCRIPT

class Parent {    static void display() {        System.out.println("Parent method
");    }}class Child extends Parent {    static void display() { // This is hiding, not overriding
Child method");    }}public class Main {    public static void main(String[] args) {

Parent p = new Child();        p.display(); // Output: Parent method (not Child!)
    }}
```

Key Differences:

Aspect	Static Method	Instance Method
Binding	Compile-time	Runtime
Belongs to	Class	Object
Can Override	No (hiding)	Yes

TL;DR: Static methods cannot be overridden; they're hidden. Compile-time binding prevents runtime polymorphism.

Keyword/Key mappings: Static methods class-level, compile-time binding, method hiding | Overriding instance methods only

Q62: How does Java achieve runtime polymorphism?

Answer: Java achieves runtime polymorphism through method overriding and dynamic method dispatch, where a parent class reference points to a child class object and the correct method is called based on the actual object type.

Polished Answer:

Mechanism:

- 1. **Method Overriding:** Child class provides its own implementation of parent's method
- 2. **Dynamic Method Dispatch:** JVM determines at runtime which method to call
- 3. **Parent Reference:** Parent class reference can point to child object
- 4. **Actual Object Type:** Determines which method executes

Example:

java

```

javascript.snippet
JAVASCRIPT

class Animal {    void sound() {        System.out.println("Animal sound
");    }}class Dog extends Animal {    @Override    void sound() {        System.out.println("
Dog barks
");    }}class Cat extends Animal {    @Override    void sound() {        System.out.println("
Cat meows");    }}public class Main {    public static void main(String[] args) {

Animal a1 = new Dog(); // Parent reference, child object        Animal a2 = new Cat();

a1.sound(); // Dog barks (runtime dispatch)        a2.sound(); // Cat meows    }}
```

Key Points:

- Requires inheritance and method overriding
- Methods must be non-static, non-private, non-final
- JVM uses method tables for dispatch
- Core mechanism for polymorphic behavior

TL;DR: Java runtime polymorphism = method overriding + dynamic method dispatch using parent reference to child object.

Keyword/Key mappings: Runtime polymorphism overriding, dynamic dispatch, parent reference, JVM

Q63: Why does Java not support multiple inheritance using classes?

Answer: Java doesn't support multiple class inheritance to avoid the diamond problem - ambiguity when two parent classes have the same method. Java uses interfaces for multiple inheritance of type.

Polished Answer:

Reason: Java's designers avoided multiple class inheritance to prevent the diamond problem, which creates ambiguity and complexity.

Diamond Problem in Java:

java

```
javascript.snippet JAVASCRIPT

// If Java allowed this: class A { void display() { System.out.println("A"); }} class B extends A
```

Java's Solution - Interfaces:

java

```
javascript.snippet JAVASCRIPT

interface A { void method(); } interface B { default void method() { System.out.println("B's method"); }} interface C { default void method() { System.out.println("C's method"); }} class D implements B, C { // Must explicitly resolve conflict @Override public void method() { B.super.method(); // Explicit choice }}
```

Key Points:

- Classes: Single inheritance only
- Interfaces: Multiple inheritance of type allowed
- Default methods: Explicit resolution required for conflicts
- Keeps language simpler and safer

TL;DR: Java avoids multiple class inheritance to prevent diamond problem; uses interfaces instead.

Keyword/Key mappings: Multiple inheritance not allowed for classes | Interfaces multiple inheritance of type | Diamond problem ambiguity

Q64: What are SOLID principles in OOPs?

Answer: SOLID is a set of five design principles: Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. They improve maintainability, extensibility, and testability.

Polished Answer:

Principle	Description	Benefit
S - Single Responsibility	Class has one reason to change	Easier testing, less coupling
O - Open-Closed	Open for extension, closed for modification	Add features without breaking existing code
L - Liskov Substitution	Subclasses substitutable for parents	Correct polymorphic behavior
I - Interface Segregation	Small focused interfaces over monolithic ones	Clients only depend on what they use
D - Dependency Inversion	Depend on abstractions, not concretions	Decoupled architecture

Examples:

SRP:

java

```

javascript.snippet
JAVASCRIPT

// Bad - Multiple responsibilities
class User {
    void saveUser() { }
    void sendEmail() { }
}

```

OCP:

java

```

javascript.snippet
JAVASCRIPT

// Open for extension via interfaces
interface Shape { double area(); }
class Circle implements Shape {
    double area() { }
}

```

TL;DR: SOLID = 5 principles for better OOP design; SRP, OCP, LSP, ISP, DIP.

Keyword/Key mappings: SOLID design principles, maintainability | SRP one responsibility | OCP extensible | LSP substitutable | ISP focused interfaces | DIP depend on abstractions

Q65: What is the Liskov Substitution Principle (LSP)?

Answer: LSP states that a subclass should be fully substitutable for its parent class without breaking the application. Swapping a subclass for its parent should not cause unexpected behavior.

Polished Answer:

Liskov Substitution Principle:

- Named after Barbara Liskov
- Subtypes must be replaceable by their base types
- A program using a base class should work correctly with any derived class
- "Is-a" relationship must be semantically correct

Violating LSP:

java

```
javascript.snippet JAVASCRIPT

class Rectangle {    protected int width, height;        void setWidth(int w) { width =
w; }    void setHeight(int h) { height = h; }    int area() { return width * height; }}
class
Square extends Rectangle {    @Override    void setWidth(int w) {        width = w;

height = w; // Forces square behavior    }    @Override    void setHeight(int h) {

height = h;        width = h; // Violates expectations!    }}// Code that breaks:void
resizeRectangle
(Rectangle r) {    r.setWidth(5);    r.setHeight(10);    // For Square, area = 100 (
10*10
), but expected 50 (5*10)}
```

Correct Design:

java

```
javascript.snippet JAVASCRIPT

interface Shape {    int area();}class Rectangle implements Shape {    int width,
height
;    public int area() { return width * height; }}class Square implements Shape {    int
side;    public int area() { return side * side; }}
```

TL;DR: LSP = subclasses must be substitutable for parents without breaking behavior or expectations.

Keyword/Key mappings: LSP substitutability, inheritance correctness, semantic is-a, behavioral subtyping

Q66: What is the Single Responsibility Principle (SRP)?

Answer: SRP states that a class should have only one reason to change, meaning it should have only one job or responsibility. This improves maintainability and testability.

Polished Answer:

Single Responsibility Principle:

- Each class has exactly one responsibility
- One reason to change = one responsibility
- Reduces coupling and increases cohesion
- Makes classes easier to understand, test, and maintain

Violating SRP:

java

```
javascript.snippet JAVASCRIPT

// Bad - This class has multiple responsibilities
class Employee {    void calculateSalary() { }
```

Correct SRP:

java

```
javascript.snippet JAVASCRIPT

class Employee {    String name;    String department;    // Employee data only}
class SalaryCalculator
{    void calculateSalary(Employee e) { }}
class EmployeeRepository {    void save(
Employee
e) { }}
class EmailService {    void sendEmail(Employee e) { }}
class ReportGenerator {
void generateReport(Employee e) { }}
```

Benefits:

- Easier to test each class independently
- Changes isolated to one class
- Better code organization
- Improved readability

TL;DR: SRP = one class, one responsibility, one reason to change.

Keyword/Key mappings: SRP single responsibility, one reason to change, separation of concerns, cohesion

Q67: What is the Open-Closed Principle (OCP)?

Answer: OCP states that software entities should be open for extension but closed for modification. New functionality should be added by extending existing code, not modifying it.

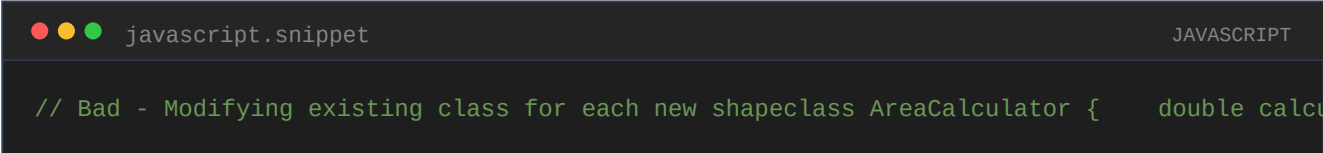
Polished Answer:

Open-Closed Principle:

- **Open for Extension:** Can add new behavior
- **Closed for Modification:** Existing tested code unchanged
- Achieved through abstraction, interfaces, inheritance

Violating OCP:

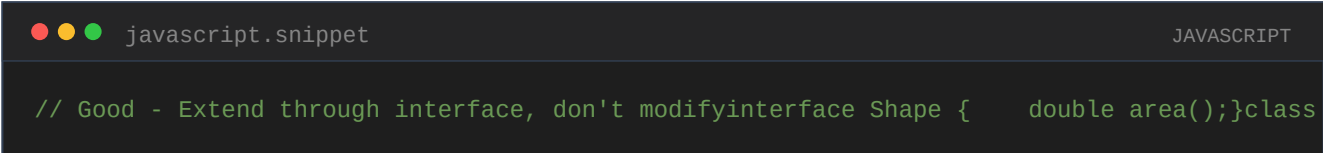
java

A screenshot of a code editor window titled 'javascript.snippet' with 'JAVASCRIPT' in the top right corner. The code inside is:

```
// Bad - Modifying existing class for each new shapeclass AreaCalculator {    double calculateArea()
```

Correct OCP:

java

A screenshot of a code editor window titled 'javascript.snippet' with 'JAVASCRIPT' in the top right corner. The code inside is:

```
// Good - Extend through interface, don't modifyinterface Shape {    double area();}class Circle im
```

TL;DR: OCP = open for extension (add new classes), closed for modification (don't change existing code).

Keyword/Key mappings: OCP extension vs modification, interfaces, abstraction, extensibility

Q68: What is the Dependency Inversion Principle (DIP)?

Answer: DIP states that high-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.

Polished Answer:

Dependency Inversion Principle:

- Decouples high-level business logic from low-level implementations
- Both depend on interfaces/abstract classes
- Enables swapping implementations easily
- Improves testability (mock objects)

Violating DIP:

java

```
javascript.snippet JAVASCRIPT
// Bad - High-level depends on low-level directly
class MySQLDatabase {    void save(String data) {
```

Correct DIP:

java

```
javascript.snippet JAVASCRIPT
// Good - Both depend on abstraction
interface Database {    void save(String data);}
class MySQLData
```

TL;DR: DIP = depend on abstractions, not concrete implementations; enables flexibility and testability.

Keyword/Key mappings: DIP abstractions, dependency injection, decoupling, interfaces

Q69: What is the Interface Segregation Principle (ISP)?

Answer: ISP states that clients should not be forced to depend on methods they do not use. Prefer small, focused interfaces over large, monolithic ones.

Polished Answer:

Interface Segregation Principle:

- Avoid "fat" interfaces with unrelated methods
- Split into smaller, specific interfaces
- Clients only implement what they need
- Reduces coupling and unnecessary dependencies

Violating ISP:

java

```
javascript.snippet JAVASCRIPT
// Bad - Fat interface forcing unwanted methods
interface Worker {    void work();    void eat();
```

Correct ISP:

java

```
javascript.snippet JAVASCRIPT
// Good - Focused interfaces
interface Workable {    void work();}
interface Eatable {    void eat();}
```

TL;DR: ISP = small focused interfaces; don't force clients to implement methods they don't need.

Keyword/Key mappings: ISP focused interfaces, avoid fat interfaces, client-specific interfaces

Q70: What is meant by static polymorphism?

Answer: Static polymorphism (compile-time polymorphism) is when an object is linked with its function or operator based on values during compile time. Achieved through method overloading and operator overloading.

Polished Answer:

Static Polymorphism:

- Resolved at compile time
- Also known as early binding
- Compiler decides which function to call
- Based on method signature (name + parameters)

Types:

1. Function/Method Overloading:

cpp

```
javascript.snippet JAVASCRIPT

void print(int x) { cout << "Integer: " << x; }void print(string s) { cout <<
"String: "
<< s; }void print(double d) { cout << "Double: " << d; }
```

1. Operator Overloading:

cpp

```
javascript.snippet JAVASCRIPT

class Complex {public:    int real, imag;    Complex operator+(const Complex& other) {
Complex result;        result.real = real + other.real;        result.imag = imag +
other
.imag;        return result;    }};
```

1. Templates (C++):

cpp

```
javascript.snippet JAVASCRIPT

template<typename T>T add(T a, T b) { return a + b; }
```

TL;DR: Static polymorphism = compile-time resolution through overloading and templates.

Keyword/Key mappings: Static polymorphism compile-time, early binding, overloading, templates

Q71: What is the difference between static polymorphism and dynamic polymorphism?

Answer: Static polymorphism resolves function calls at compile time using overloading.

Dynamic polymorphism resolves at runtime using overriding and virtual functions based on actual object type.

Polished Answer:

Aspect	Static Polymorphism	Dynamic Polymorphism
Binding	Compile-time (early)	Runtime (late)
Achieved By	Overloading, templates	Overriding, virtual functions
Decision Maker	Compiler	JVM/Runtime
Performance	Faster	Slightly slower (vtable lookup)
Flexibility	Less flexible	More flexible
Relationship	Same class	Parent-child classes
Method Signature	Different parameters	Same parameters

Static Example:

cpp

```

javascript.snippet
JAVASCRIPT

class Calculator {
  int add(int a, int b) { return a + b; }
  double add(double a, double b) { return a + b; }
};

```

Dynamic Example:

cpp

```

javascript.snippet
JAVASCRIPT

class Animal {public:
  virtual void sound() { }};
class Dog : public Animal {
  void sound() override { cout << "Bark"; }
};

```

TL;DR: Static = compile-time, overloading, same class; Dynamic = runtime, overriding, inheritance.

Keyword/Key mappings: Static compile-time, overloading | Dynamic runtime, overriding, virtual

Q72: How does C++ support Polymorphism?

Answer: C++ supports compile-time polymorphism through templates, function overloading, and default arguments. Runtime polymorphism is supported through virtual functions and

function overriding.

Polished Answer:

1. Compile-Time Polymorphism in C++:

Function Overloading:

cpp

```
javascript.snippet JAVASCRIPT

int add(int a, int b) { return a + b; }double add(double a, double b) { return a + b; }
```

Operator Overloading:

cpp

```
javascript.snippet JAVASCRIPT

class Vector {public:    Vector operator+(const Vector& v) { }};
```

Templates:

cpp

```
javascript.snippet JAVASCRIPT

template<typename T>T max(T a, T b) {    return (a > b) ? a : b;}
```

Default Arguments:

cpp

```
javascript.snippet JAVASCRIPT

int multiply(int a, int b = 2) { return a * b; }// Can call multiply(5) or multiply(5,
3
)
```

2. Runtime Polymorphism in C++:

cpp

```
javascript.snippet JAVASCRIPT

class Base {public:    virtual void print() {        cout << "Base print";    }};class
Derived
: public Base {public:    void print() override {        cout << "Derived print";
    }};
Base* b = new Derived();b->print(); // Derived print (runtime dispatch)
```

TL;DR: C++ supports both compile-time (templates, overloading) and runtime (virtual functions) polymorphism.

Keyword/Key mappings: C++ polymorphism templates, overloading, virtual functions, override

Q73: What is a subclass and superclass?

Answer: A subclass (child class) is a class that inherits from another class. A superclass (parent/base class) is the class being inherited from. The subclass acquires members of the superclass.

Polished Answer:

Superclass (Parent/Base Class):

- The class being inherited from
- Provides common properties and methods
- Can be extended by multiple subclasses

Subclass (Child/Derived Class):

- The class that inherits from superclass
- Acquires superclass members
- Can add its own members
- Can override inherited methods

Example:

java

```
javascript.snippet                                JAVASCRIPT

class Vehicle {                                // Superclass    String brand;    void start() {                                System
.
out.println("Vehicle started
");    }}class Car extends Vehicle { // Subclass    int wheels = 4;    @Override    void start
Car started
");    }    void honk() { // New method in subclass    System.out.println("
Beep beep!");    }}
```

Key Points:

- Subclass inherits all public and protected members
- Private members are not directly accessible
- Subclass can extend or modify superclass behavior
- Inheritance establishes hierarchy

TL;DR: Superclass = parent class being inherited; Subclass = child class that inherits from parent.

Keyword/Key mappings: Superclass parent, base class | Subclass child, derived class, extends

Q74: What is Method Hiding in Java?

Answer: Method hiding occurs when a subclass defines a static method with the same signature as a static method in the parent class. The parent's method is hidden, not overridden. Method resolution happens at compile time.

Polished Answer:

Method Hiding vs Overriding:

Aspect	Method Hiding	Method Overriding
Method Type	Static methods	Instance methods
Binding	Compile-time	Runtime
Resolution	Based on reference type	Based on object type
Keyword	No special keyword needed	@Override

Method Hiding Example:

```

class Parent {    static void display() {        System.out.println("Parent static
method
");    }}class Child extends Parent {    static void display() { // Hides parent's method
Child static method");    }}public class Main {    public static void main(String[]
args
) {        Parent p = new Child();        p.display(); // Output: Parent static method

(compile-time binding)        Child c = new Child();        c.display(); //
Output
: Child static method    }}

```

Key Points:

- Static methods cannot be overridden, only hidden
- Method resolution based on reference type, not object type
- No dynamic dispatch for static methods
- Often confused with overriding

TL;DR: Method hiding = static method with same signature in child; resolved at compile time, not runtime.

Keyword/Key mappings: Method hiding static methods, compile-time binding, reference type

Q75: What are some other programming paradigms besides OOPs?

Answer: Other paradigms include Procedural, Functional, Logical, Parallel, and Database programming. They're classified under Imperative (how to execute) and Declarative (what to execute) paradigms.

Polished Answer:**Programming Paradigms Classification:****1. Imperative Paradigms (How to execute):**

- **Procedural:** Sequential steps (C, Pascal)
 - **Object-Oriented:** Objects with data and behavior (Java, C++)
 - **Parallel:** Concurrent execution (CUDA, OpenMP)
-

2. Declarative Paradigms (What to execute):

- **Functional:** Pure functions, immutability (Haskell, Scala)
- **Logical:** Formal logic rules (Prolog)
- **Database:** Query-based (SQL)

Examples:

Procedural:

C

```
javascript.snippet JAVASCRIPT
int sum = 0;for (int i = 0; i < 10; i++) {    sum += i;}
```

Functional:

haskell

```
javascript.snippet JAVASCRIPT
-- No loops, use recursionsum [1..10]
```

Logical:

prolog

```
javascript.snippet JAVASCRIPT
parent(john, mary).parent(mary, alice).grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

Key Points:

- OOP is one of many paradigms
- Languages can support multiple paradigms
- Python supports OOP, functional, and procedural
- Choice depends on problem requirements

TL;DR: Paradigms = Imperative (Procedural, OOP, Parallel) and Declarative (Functional, Logical, Database).

Keyword/Key mappings: Paradigms imperative, declarative | Imperative procedural, OOP, parallel | Declarative functional, logical, database

Q76: How do you apply OOPs in a real-world design problem?

Answer: Apply OOPs by identifying classes (nouns), using abstraction for interfaces, polymorphism for flexible behavior, encapsulation for data protection, and composition for object relationships.

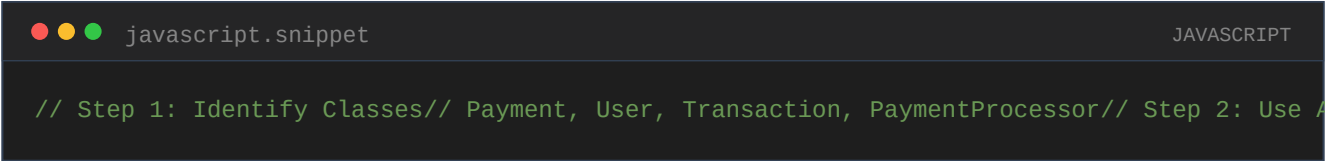
Polished Answer:

Step-by-Step Approach:

1. **Identify Classes:** Find nouns in requirements (User, Payment, Order)
2. **Define Relationships:** Determine is-a (inheritance) vs has-a (composition)
3. **Apply Abstraction:** Define interfaces for common behaviors
4. **Use Polymorphism:** Allow different implementations of same interface
5. **Encapsulate Data:** Make fields private with controlled access

Example - Payment Processing System:

java



```
// Step 1: Identify Classes// Payment, User, Transaction, PaymentProcessor// Step 2: Use Abstraction
```

TL;DR: Real-world OOP design = identify classes define relationships apply abstraction use polymorphism encapsulate data.

Keyword/Key mappings: Real-world OOP class identification, relationships, abstraction, polymorphism, composition

Q77: What are the main features of OOPs?

Answer: The main features (pillars) of OOPs are Encapsulation, Data Abstraction, Inheritance, and Polymorphism.

Polished Answer:

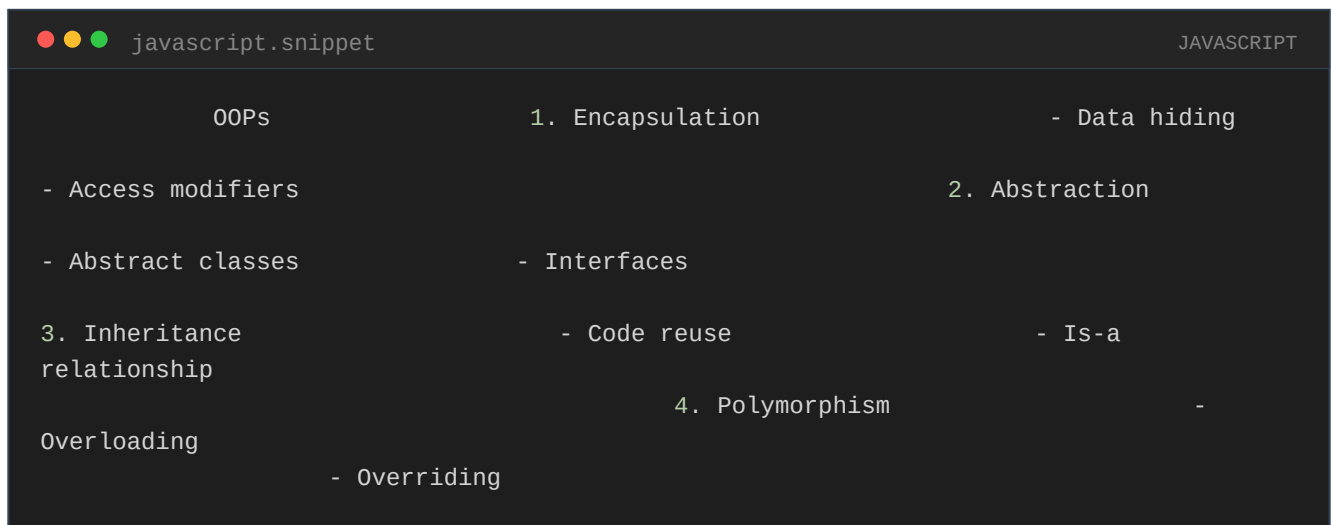
Four Pillars of OOPs:

1. **Encapsulation:** Bundles data and methods, hides internal state
 2. **Abstraction:** Shows essential features, hides implementation details
 3. **Inheritance:** Allows classes to acquire properties from other classes
-

4. Polymorphism: Same interface, different behaviors based on context

Visual Representation:

text



Example Demonstrating All Four:

cpp

```
javascript.snippet JAVASCRIPT
// Encapsulation + Abstraction
class Shape {
protected:
    string color; // Encapsulated data
public:
```

TL;DR: Four pillars = Encapsulation, Abstraction, Inheritance, Polymorphism - foundation of OOPs.

Keyword/Key mappings: Features encapsulation, abstraction, inheritance, polymorphism, pillars

Q78: How much memory does a class occupy?

Answer: Classes do not consume memory. They are just blueprints or templates. Objects created from classes consume memory when they initialize class members and methods.

Polished Answer:

Memory Allocation:

- **Class:** No memory allocated (just a template)
- **Object:** Memory allocated when instantiated
- **Size depends on:** Instance variables, not methods

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Student {    string name;        // Memory for each object    int age;        //
Memory
    for each object    double marks;    // Memory for each object    void display() {

// No per-object memory (shared code)    cout << name;    };// Student class - 0
bytes
    (no object)Student s1; // Memory allocated: sizeof(string) + sizeof(int) + sizeof(
double
    )Student s2; // Separate memory allocation
```

Memory Allocation Details:

- **Stack:** Local objects allocated here
- **Heap:** Dynamically allocated objects (new)
- **Static/Global:** Global objects
- **Code segment:** Methods are shared, not duplicated per object

Size Calculation:

cpp

```
javascript.snippet JAVASCRIPT

class Example {    char c;        // 1 byte    int i;        // 4 bytes    double d;    // 8
bytes
};// sizeof(Example) = 16 bytes (due to padding/alignment)
```

TL;DR: Class = no memory; Object = memory for data members only; methods shared across all objects.

Keyword/Key mappings: Class no memory, blueprint | Object memory for data, shared methods

Q79: Is it always necessary to create objects from a class?

Answer: No. Objects are necessary for non-static methods, but static methods can be called directly using the class name without creating objects.

Polished Answer:

When Objects Are Needed:

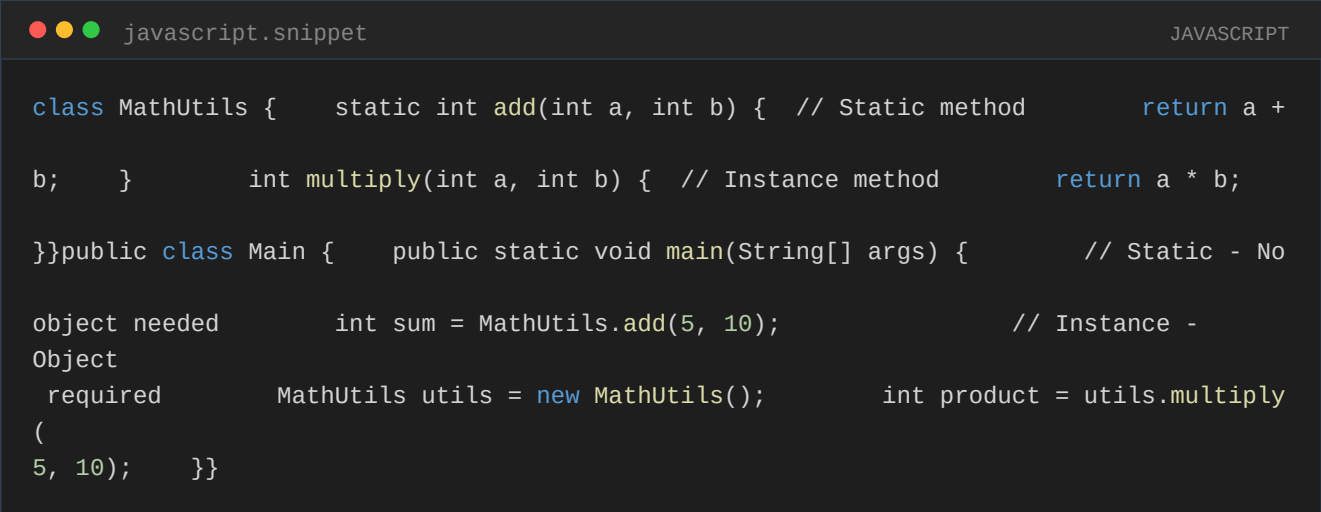
- Accessing non-static (instance) members
- Using instance-specific data
- Creating multiple independent instances

When Objects Are NOT Needed:

- Calling static methods
- Accessing static variables
- Utility classes with only static members

Example:

java

A screenshot of a code editor window titled 'javascript.snippet' with 'JAVASCRIPT' in the top right corner. The code is in Java and defines two classes: MathUtils and Main. MathUtils has a static method 'add' and an instance method 'multiply'. Main has a static 'main' method that demonstrates using both types of methods. Comments in the code indicate when an object is needed or not.

```
class MathUtils {    static int add(int a, int b) { // Static method        return a + b;    }    int multiply(int a, int b) { // Instance method        return a * b;    }}public class Main {    public static void main(String[] args) {        // Static - No object needed        int sum = MathUtils.add(5, 10);        // Instance - Object required        MathUtils utils = new MathUtils();        int product = utils.multiply(5, 10);    }}}
```

TL;DR: Static methods = no object needed; Instance methods = object required.

Keyword/Key mappings: Static methods class name directly | Instance methods object required

Q80: What are access specifiers? What is their significance in OOPs?

Answer: Access specifiers control accessibility of class members. Types: public (accessible everywhere), private (only within class), protected (within class and derived classes). They're crucial for encapsulation and data hiding.

Polished Answer:

Access Specifiers:

Specifier	Within Class	Derived Class	Outside Class
public	v	v	v
protected	v	v	
private	v		

Example:

cpp

```

class BankAccount {private:      // Only accessible within class    double balance;

string accountNumber;    protected:      // Accessible in class and derived classes
string
accountType;    public:      // Accessible everywhere    void deposit(double amount
)
{        balance += amount; // Private access within class    }    double getBalance
()
{        return balance;    }};

```

Significance:

- **Encapsulation:** Hide internal implementation
- **Data Security:** Prevent unauthorized access
- **Controlled Access:** Through public methods (getters/setters)
- **Interface Design:** Define what users can access

TL;DR: Access specifiers = public, private, protected; control visibility; essential for encapsulation.

Keyword/Key mappings: public everywhere | private class only | protected class + derived

Q81: What are friend functions and friend classes in C++?

Answer: Friend functions are non-member functions granted access to private and protected members. Friend classes are classes whose members can access private/protected members of another class.

Polished Answer:

Friend Function:

cpp

```
javascript.snippet JAVASCRIPT

class Box {private:    int length;public:    Box() : length(10) { }    friend void printLength
(Box b); // Friend declaration};// Friend function - not a member, but can access
privatevoid
printLength(Box b) {    cout << b.length; // Can access private member!}
```

Friend Class:

cpp

```
javascript.snippet JAVASCRIPT

class A {private:    int secret;public:    friend class B; // Class B is a friend};
class
B {public:    void accessA(A obj) {        cout << obj.secret; // Can access A's
private
members    }};
```

Important Rules:

- Friendship is not mutual (A is friend of B B is friend of A)
- Friendship is not inherited
- Friendship is not transitive
- Breaks encapsulation (use sparingly)
- Often used for operator overloading

TL;DR: Friend = special access to private members for non-member functions or other classes.

Keyword/Key mappings: Friend function non-member, private access | Friend class full access, not mutual/inherited

Q82: Can we overload the constructor in a class?

Answer: Yes, constructors can be overloaded in a class. Constructor overloading is done when we want constructors with different parameters (number and type).

Polished Answer:

Constructor Overloading:

- Multiple constructors with different parameters
- Same name as class, different signatures
- Compiler selects based on arguments passed

Example:

cpp

```

class Student {    string name;    int age;    double marks;    public:    // Default constructor
    Student() {        name = "Unknown";        age = 0;        marks = 0;    }
//
// Parameterized - 1 parameter    Student(string n) {        name = n;        age = 0;
marks = 0;    }    // Parameterized - 2 parameters    Student(string n, int a) {
name = n;        age = a;        marks = 0;    }    // Parameterized - 3 parameters
Student(string n, int a, double m) {        name = n;        age = a;        marks = m;
};
// Usage
Student s1;    // Default
Student s2("John");    // 1 parameter
Student s3("John", 20);    // 2 parameters
Student s4("John", 20, 85.5);    // 3 parameters

```

TL;DR: Constructor overloading = multiple constructors with different parameter lists in same class.

Keyword/Key mappings: Constructor overloading different parameters, same class, compile-time

Q83: Can we overload the destructor in a class?

Answer: No, a destructor cannot be overloaded. There can only be one destructor in a class. It takes no parameters and has no return type.

Polished Answer:

Why Destructor Cannot Be Overloaded:

- Destructor takes no parameters
- No way to distinguish between overloaded versions
- Only called automatically when object is destroyed
- Overloading requires different parameter lists

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Example {public:    ~Example() { // Only one destructor possible    cout <<
"Destructor called"
;    }    // NOT ALLOWED - can't overload destructor    // ~Example(int x) { }    //
ERROR
!    // Constructors CAN be overloaded    Example() { }    Example(int x) { };
```

Key Points:

- Destructor has no parameters, no return type
- Only one destructor per class
- Automatically called on object destruction
- Cannot be overloaded, inherited, or static

TL;DR: Destructor cannot be overloaded; only one per class with no parameters.

Keyword/Key mappings: Destructor no overloading, no parameters, one per class, automatic

Q84: What is the difference between a structure and a class in C++?

Answer: In C++, both are user-defined data types, but structures have public access by default while classes have private access by default. Syntax differs: struct vs class keywords.

Polished Answer:

Aspect	Structure (struct)	Class
Default Access	public	private
Default Inheritance	public	private
Keyword	struct	class
Usage	Simple data containers	Complex objects with behavior

Example:

cpp

```
javascript.snippet                                JAVASCRIPT

struct Point {    int x;  // Public by default    int y;};
class Student {    string
name
;  // Private by default!public:    void setName(string n) { name = n; };
```

Key Points:

- Both can have constructors, destructors, methods
- Both support inheritance
- Difference is only default access level
- struct commonly used for POD (Plain Old Data)

TL;DR: struct = public by default; class = private by default; otherwise nearly identical in C++.

Keyword/Key mappings: struct public default, data container | class private default, encapsulation

Q85: What is the difference between shallow copy and deep copy?

Answer: Shallow copy copies references/pointers, so both objects share the same memory for dynamic data. Deep copy creates independent copies of all data including dynamically allocated memory.

Polished Answer:

Aspect	Shallow Copy	Deep Copy
Memory	Shares memory for dynamic data	Allocates new memory for each object
Independence	Objects are dependent	Objects are fully independent
Changes	Affect both objects	Don't affect each other
Performance	Faster	Slower (more memory/time)
Safety	Can cause dangling pointers	Safer for complex objects

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Student {public:    string* name; // Dynamically allocated // Shallow
Copy
    (default copy constructor)    Student(const Student& obj) {        name = obj.name;
//
    Both point to same memory!    } // Deep Copy    Student(const Student& obj) {
name = new string(*obj.name); // New memory allocation    }};
```

Problem with Shallow Copy: If one object is destroyed, the other has a dangling pointer.

TL;DR: Shallow copy = shared memory (dangerous); Deep copy = independent memory (safe for dynamic data).

Keyword/Key mappings: Shallow copy shared references, same memory | Deep copy independent copy, new memory allocation

Q86: What is the Diamond Problem? How can it be resolved?

Answer: The Diamond Problem occurs in multiple inheritance when a class inherits the same base class through two different parent classes, causing ambiguity. Resolved in C++ using virtual inheritance; Java avoids it by not supporting multiple class inheritance.

Polished Answer:

Diamond Problem:

text

```
javascript.snippet JAVASCRIPT

A (Base Class)    / \    B    C (Both inherit from A)    \ /    D (Inherits from
both B and C)
```

Problem: Class D has two copies of class A's members, causing ambiguity.

C++ Solution - Virtual Inheritance:

cpp

```
javascript.snippet JAVASCRIPT

class A {public:    void display() { }};class B : virtual public A { }; // Virtual inheritance
class C : virtual public A { }; // Virtual inheritance
class D : public B, public C { }; //

Now only ONE copy of A
```

Java Solution:

Java disallows multiple class inheritance entirely, eliminating the problem. Multiple inheritance is achieved through interfaces.

Key Points:

- Virtual inheritance ensures only one shared base class instance
- Increases object size slightly (vtable pointers)
- Common interview question for C++ developers

TL;DR: Diamond problem = ambiguous multiple inheritance; Solution = virtual inheritance (C++) or interfaces (Java).

Keyword/Key mappings: Diamond problem multiple inheritance, ambiguity | Virtual inheritance single shared base instance

Q87: What are Virtual Functions and Pure Virtual Functions?

Answer: Virtual functions are methods that can be overridden in derived classes, enabling runtime polymorphism. Pure virtual functions have no implementation (declared with = 0) and make the class abstract; they must be overridden by derived classes.

Polished Answer:

Virtual Function:

- Declared with `virtual` keyword in base class
- Provides default implementation
- Can be overridden in derived classes
- Enables dynamic dispatch based on actual object type

cpp

```
javascript.snippet JAVASCRIPT

class Animal {public:    virtual void sound() {           // Virtual function
cout
    << "Animal makes sound";    }};class Dog : public Animal {public:    void sound()
override
{           // Override        cout << "Dog barks";    }};
```

Pure Virtual Function:

- Declared with = 0
- No implementation in base class
- Makes the class abstract (cannot be instantiated)
- Must be overridden by concrete derived classes

cpp

```
javascript.snippet JAVASCRIPT

class Shape {public:    virtual double area() = 0; // Pure virtual function};class
Circle
: public Shape {    double area() override { return 3.14 * r * r; } // Must implement};
```

Key Points:

- Virtual functions are the mechanism for runtime polymorphism
- Pure virtual functions define interfaces
- Java: all non-static, non-final, non-private methods are virtual
- Python: all methods are virtual by default

TL;DR: Virtual function = overridable with default implementation; Pure virtual = no implementation, must override, makes class abstract.

Keyword/Key mappings: Virtual override, dynamic dispatch, runtime polymorphism | Pure virtual abstract, = 0, interface

Q88: What is Method Resolution Order (MRO) in Python?

Answer: MRO defines the order in which Python searches for methods in class hierarchies with multiple inheritance. It uses the C3 linearization algorithm for consistent, predictable lookup.

Polished Answer:

MRO: Determines method lookup order when multiple inheritance creates ambiguity.

Example:

python

```
class A:
    def method(self):
        print("A")
class B(A):
    def method(self):
        print("B")
class C(A):
    def method(self):
        print("C")
class D(B, C): # Multiple inheritance
    pass
print(D.mro()) # Output: [D, B, C, A, object]
d = D()
d.method() # Output: B (B comes before C in MRO)
```

C3 Linearization Algorithm:

- Ensures consistent ordering
- Preserves local precedence
- Avoids ambiguous hierarchies
- Guarantees each class appears before its parents

Key Points:

- Python searches methods in MRO order
- `super()` follows MRO
- Accessible via `ClassName.mro()` or `ClassName.__mro__`

TL;DR: MRO = method lookup order in multiple inheritance, calculated by C3 algorithm.

Keyword/Key mappings: MRO C3 linearization, method lookup, multiple inheritance | `super()` follows MRO

Q89: Why should a base class destructor be virtual in C++?

Answer: A virtual destructor ensures proper cleanup when deleting a derived object through a base class pointer. Without it, only the base destructor runs, causing memory leaks.

Polished Answer:

Problem Without Virtual Destructor:

cpp

```
javascript.snippet JAVASCRIPT

class Base {public:    ~Base() { cout << "Base destructor"; }};class Derived : public
Base
{    int* data; // Dynamically allocatedpublic:    Derived() { data = new int[100]; }

~Derived() {        delete[] data; // Never called!        cout <<
"Derived destructor"
;    }};Base* ptr = new Derived();delete ptr; // Only Base destructor called MEMORY
LEAK
!
```

Solution:

cpp

```
javascript.snippet JAVASCRIPT

class Base {public:    virtual ~Base() { cout << "Base destructor"; } // Virtual
destructor
};
```

Key Points:

- Rule: If a class has virtual functions, it needs virtual destructor
- Ensures proper cleanup in polymorphic hierarchies
- Slight overhead due to vtable
- Critical for memory management in C++

TL;DR: Virtual destructor = proper cleanup when deleting derived objects through base pointer; prevents memory leaks.

Keyword/Key mappings: Virtual destructor proper cleanup, polymorphic delete, memory leak prevention

Q90: What is the purpose of the virtual keyword in C++?

Answer: The virtual keyword enables runtime polymorphism by allowing derived classes to override base class methods. It ensures the correct function is called based on the actual object type, not the pointer type.

Polished Answer:

Purpose of virtual:

1. **Method Overriding:** Allows derived classes to provide specific implementations
2. **Dynamic Dispatch:** Correct method called at runtime based on actual object
3. **Polymorphic Behavior:** Same interface, different behaviors

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Shape {public:    virtual void draw() { cout << "Drawing Shape"; }};class Circle
:
public Shape {public:    void draw() override { cout << "Drawing Circle"; }};class
Rectangle
: public Shape {public:    void draw() override { cout << "Drawing Rectangle"; }};
Shape
* shapes[] = {new Circle(), new Rectangle()};shapes[0]->draw(); // Drawing Circle (
virtual
dispatch)shapes[1]->draw(); // Drawing Rectangle
```

Without virtual:

cpp

```
javascript.snippet JAVASCRIPT

// Without virtual keywordShape* shape = new Circle();shape->draw(); // Drawing Shape (wrong! stat
```

TL;DR: virtual enables dynamic dispatch; correct method called based on actual object type at runtime.

Keyword/Key mappings: virtual dynamic dispatch, runtime polymorphism, override, vtable

Q91: What are the advantages and disadvantages of OOPs?

Answer: Advantages: code reusability, maintainability, data security, modularity.

Disadvantages: steeper learning curve, complex design, more memory/time overhead.

Polished Answer:

Advantages:

- **Code Reusability:** Inheritance allows reuse of existing code
- **Maintainability:** Modular design makes updates and fixes easier
- **Data Security:** Encapsulation protects data through access modifiers

- **Modularity:** Classes represent separate units for better organization
- **Scalability:** Easier to extend for large applications
- **DRY Principle:** Write once, reuse multiple times
- **Real-world Modeling:** Objects mirror real-world entities

Disadvantages:

- **Learning Curve:** Concepts like inheritance, polymorphism are complex
- **Design Overhead:** Requires careful planning and design
- **Performance:** Objects and virtual functions add overhead
- **Memory Usage:** Objects consume more memory than simple data structures
- **Not Always Necessary:** Overkill for small, simple programs

TL;DR: OOP advantages = reusable, maintainable, secure code; Disadvantages = complex, slower, more memory.

Keyword/Key mappings: Advantages reusability, encapsulation, modularity | Disadvantages complexity, overhead, learning curve

Q92: What is the difference between Procedural Programming and OOP?

Answer: Procedural programming focuses on functions with separate data, less secure, better for small programs. OOP focuses on objects that bundle data and methods, more secure through encapsulation, better for large applications.

Polished Answer:

Aspect	Procedural Programming	OOP
Focus	Functions	Objects
Data & Functions	Separate	Bundled together
Security	Less secure	More secure (encapsulation)
Approach	Top-down	Bottom-up
Reusability	Limited	High (inheritance)
Scalability	Difficult for large apps	Better for large apps
Examples	C, Pascal	Java, C++, Python

Procedural Example:

C

```
javascript.snippet JAVASCRIPT
// Data separate from functions
float radius;
float area(float r) { return 3.14 * r * r; }
```

OOP Example:

cpp

```
javascript.snippet JAVASCRIPT
// Data and methods together
class Circle { float radius; // Data
public: float area() { // M
```

Key Points:

- OOP organizes code around real-world entities
- Procedural organizes code as sequential steps
- OOP provides better abstraction and modularity
- Both have their use cases

TL;DR: Procedural = functions + separate data; OOP = objects bundling data + methods together.

Keyword/Key mappings: Procedural functions, top-down, separate data | OOP objects, bottom-up, bundled data+methods

Q93: What is the need for OOPs? Why is it preferred?

Answer: OOPs helps users understand software easily, increases readability and maintainability, and enables writing and managing very large software efficiently through modular design.

Polished Answer:

Why OOPs is Needed:

1. **Complexity Management:** Large systems become manageable by breaking them into objects
2. **Real-world Modeling:** Maps directly to how we perceive the world
3. **Team Collaboration:** Modular classes allow multiple developers to work simultaneously

- 4. **Maintenance:** Easier to find and fix bugs in isolated classes
- 5. **Extensibility:** New features can be added without breaking existing code

Example:

cpp

```
javascript.snippet JAVASCRIPT
// Procedural approach
float calculateArea(float length, float width) {    return length * width;}
```

TL;DR: OOPs needed for managing complexity, modeling real-world, improving maintainability and team collaboration.

Keyword/Key mappings: Need complexity management, real-world modeling, maintainability, scalability

Q94: What is meant by Structured Programming?

Answer: Structured Programming is a programming method with organized control flow using blocks containing rules and definitive control flows like if/then/else, while/for loops, block structures, and subroutines.

Polished Answer:

Characteristics:

- Clear control structures (sequence, selection, iteration)
- No goto statements (or minimal use)
- Modular blocks (functions/procedures)
- Top-down design approach
- Single entry, single exit for blocks

Control Structures:

c

```
javascript.snippet JAVASCRIPT
// Sequence
int a = 5; int b = 10;
// Selection
if (a > b) {    printf("a is larger");} else {    printf("b is larger");}
```

Key Points:

- Nearly all paradigms (including OOP) include structured programming

- Focuses on clear, readable control flow
- Basis for all modern programming

TL;DR: Structured programming = organized control flow with blocks, loops, and conditionals without goto.

Keyword/Key mappings: Structured programming control flow, blocks, loops, conditionals

Q95: What is Garbage Collection in OOPs?

Answer: Garbage collection is the mechanism of automatically managing memory by freeing up memory occupied by objects that are no longer needed, preventing memory-related errors.

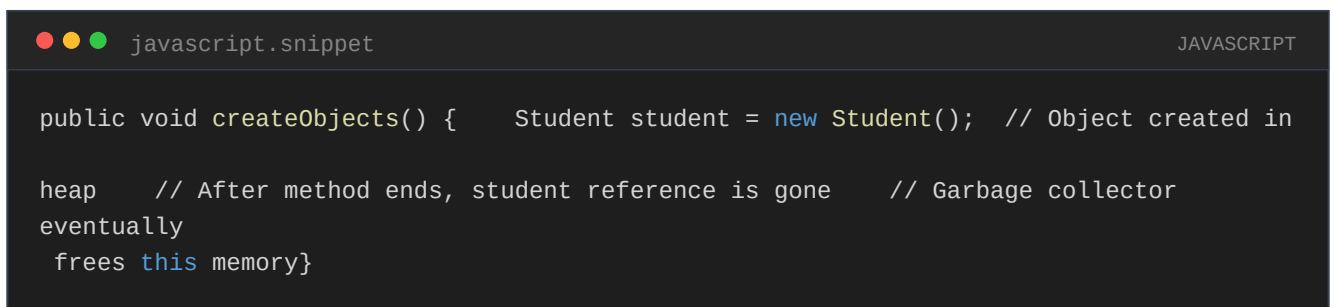
Polished Answer:

Garbage Collection:

- Automatic memory management mechanism
- Identifies and removes objects no longer referenced
- Prevents memory leaks and dangling pointers
- Found in Java, Python, C# (not C++ - manual management)

How It Works (Java Example):

java



```
public void createObjects() {    Student student = new Student(); // Object created in
heap    // After method ends, student reference is gone    // Garbage collector
eventually
    frees this memory}
```

Key Points:

- **C++:** Manual memory management (new/delete, smart pointers)
- **Java:** Automatic GC (mark and sweep, generational)
- **Python:** Reference counting + GC for cycles
- Not deterministic - cannot predict exactly when GC runs

Benefits:

- Prevents memory leaks

- Reduces programmer burden
- Automatic cleanup of unreachable objects

TL;DR: Garbage collection = automatic memory management that frees unused objects; Java/Python have it, C++ uses manual management.

Keyword/Key mappings: Garbage collection automatic memory, unreachable objects, memory leak prevention

Q96: What is exception handling in OOPs?

Answer: Exception handling is the mechanism for identifying undesirable states a program might reach and specifying desirable outcomes, preventing program crashes. Try-catch is the most common method.

Polished Answer:

Exception Handling:

- Detects and handles runtime errors gracefully
- Prevents program termination on unexpected inputs
- Provides alternate execution paths for error conditions

Try-Catch-Finally Structure:

java

```
javascript.snippet JAVASCRIPT

try {    // Code that might throw exception    int result = 10 / 0;    // Throws
ArithmeticException
} catch (ArithmeticException e) {    // Handle specific exception    System.out.println
("
Cannot divide by zero!");} finally {    // Always executed    System.out.println("
Cleanup
code");}
```

Key Components:

- **try:** Contains code that might throw exception
- **catch:** Handles specific exception types
- **finally:** Always executes (cleanup)
- **throw/throws:** Explicitly throw or declare exceptions

Benefits:

- Graceful error handling
- Separates error handling from main logic
- Prevents system crashes
- Provides meaningful error messages

TL;DR: Exception handling = try-catch mechanism to handle runtime errors gracefully without crashing.

Keyword/Key mappings: Exception handling try-catch, runtime errors, graceful recovery | finally always executes

Q97: What is Method Overloading? Provide an example.

Answer: Method overloading is compile-time polymorphism where multiple methods have the same name but different parameter lists (different number, types, or order of parameters).

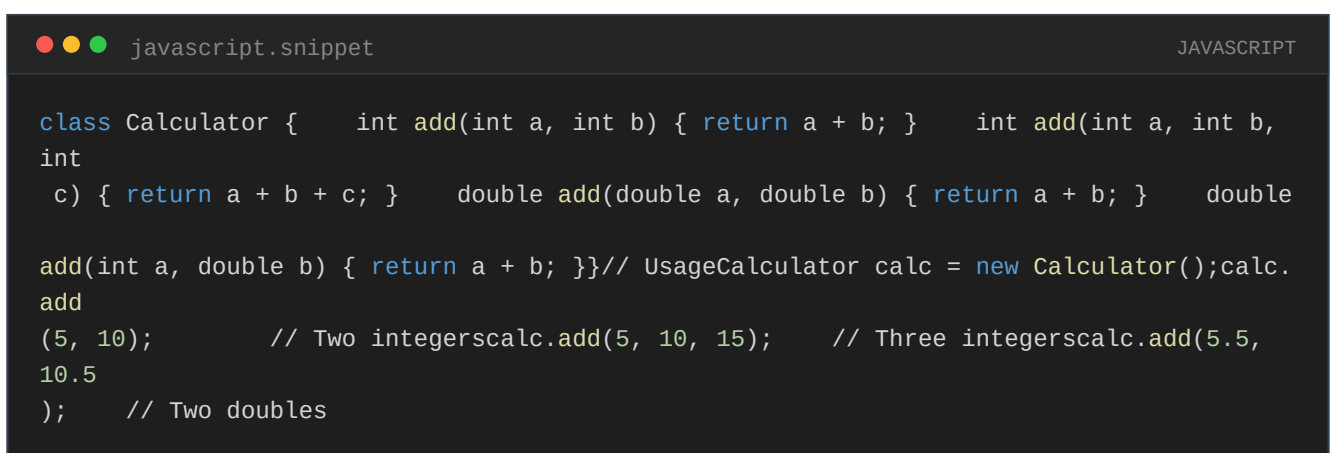
Polished Answer:

Method Overloading Rules:

- Same method name
- Different parameter lists (count, type, or order)
- Return type alone doesn't distinguish methods
- Resolved at compile time

Example:

java



```

class Calculator {
  int add(int a, int b) { return a + b; }
  int add(int a, int b, int c) { return a + b + c; }
  double add(double a, double b) { return a + b; }
  double add(int a, double b) { return a + b; }}
// Usage
Calculator calc = new Calculator();
calc.add(5, 10); // Two integers
calc.add(5, 10, 15); // Three integers
calc.add(5.5, 10.5); // Two doubles

```

TL;DR: Overloading = same method name, different parameters, resolved at compile time.

Keyword/Key mappings: Overloading same name, different parameters, compile-time,

Q98: What is Method Overriding? Provide an example.

Answer: Method overriding is runtime polymorphism where a derived class provides its own implementation of a method defined in the parent class with the same signature.

Polished Answer:

Method Overriding Rules:

- Same method name and signature as parent
- Inheritance required (parent-child relationship)
- Parent method must be virtual (C++) or non-final/non-static (Java)
- Return type must be same or covariant

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Animal {public:    virtual void sound() { cout << "Animal makes sound"; }};class Dog
: public Animal {public:    void sound() override { cout << "Dog barks"; }};class Cat
:
public Animal {public:    void sound() override { cout << "Cat meows"; }};Animal*
animal1
= new Dog();Animal* animal2 = new Cat();animal1->sound(); // Dog barksanimal2->sound
();
// Cat meows
```

Key Points:

- Uses override keyword (C++11+) for clarity
- Resolved at runtime based on actual object type
- Enables polymorphic behavior

TL;DR: Overriding = same method signature, child provides own implementation, runtime polymorphism.

Keyword/Key mappings: Overriding virtual, override, runtime polymorphism, inheritance, same signature

Q99: What is a Pure Virtual Function?

Answer: A pure virtual function is a virtual function declared with `= 0` in the base class, has no implementation, and must be overridden by derived classes. It makes the class abstract.

Polished Answer:

Characteristics:

- Declared with `= 0`
- No body in base class
- Makes the class abstract (cannot instantiate)
- Derived classes must implement it (unless also abstract)

Example:

cpp

```
javascript.snippet JAVASCRIPT

class Shape {public:    virtual double area() = 0; // Pure virtual    virtual void
draw
() = 0;    // Pure virtual};class Circle : public Shape {    double radius;public:
double
    area() override { return 3.14    radius    radius; }    void draw() override { cout <<
"Drawing circle"
; }}; // Shape s; // ERROR! Cannot instantiate abstract classShape* s = new Circle();
//
    OK, pointer to abstract class
```

Python Equivalent:

python

```
javascript.snippet JAVASCRIPT

from abc import ABC, abstractmethodclass Shape(ABC):    @abstractmethod    def area(
self
):        pass # Abstract method
```

TL;DR: Pure virtual function = abstract method with `= 0`, no implementation, makes class abstract.

Keyword/Key mappings: Pure virtual abstract, `= 0`, interface, must override

Q100: How is Data Abstraction accomplished?

Answer: Data abstraction is accomplished using abstract classes and interfaces, which declare methods without providing implementations, hiding irrelevant details from users.

Polished Answer:

Achieving Abstraction:

1. **Abstract Classes:** Classes with at least one pure virtual function
2. **Interfaces:** Special classes with only method declarations (Java/C#)
3. **Access Modifiers:** Private members hide implementation details
4. **Header Files (C++):** Declaration separated from implementation

Example:

cpp

```
javascript.snippet                                JAVASCRIPT
// Abstract class for abstractionclass Database {public:    virtual void connect() = 0;    // User k
```

Benefits:

- Simplifies user interaction
- Hides implementation complexity
- Allows changing implementation without affecting users

TL;DR: Abstraction achieved through abstract classes/interfaces that hide implementation details from users.

Keyword/Key mappings: Abstraction abstract class, interface, hide implementation, pure virtual

Summary Table: Question Importance & Distribution

Tier	Questions	Key Topics Covered
Top 10 (Q1-Q10)	4 Pillars, Class vs Object, Polymorphism types, Encapsulation, Inheritance, Abstraction vs Encapsulation, Overloading vs Overriding, Constructors, Shallow vs Deep Copy	Must-know fundamentals
Top 25 (Q11-Q25)	Access specifiers, Abstract class vs Interface, Diamond problem, Virtual functions, Association/Aggregation/Composition, Destructors, Inheritance vs Composition, Copy constructors, Friend functions, struct vs class, MRO, Virtual destructors, OOP advantages/disadvantages	Core OOP concepts
Top 50 (Q26-Q50)	Structured programming, Garbage collection, Exception handling, Overloading/Overriding examples, Pure virtual functions, Abstraction methods, Dynamic polymorphism, SOLID principles (all 5), Static vs Dynamic polymorphism, Subclass/Superclass, Method hiding, Programming paradigms	Advanced concepts & design principles
Top 100 (Q51-Q100)	Comprehensive coverage of all above with detailed examples, code samples, edge cases, Python/Java/C++ specifics	Complete OOP mastery

Final Preparation Tips

- 1. Master Top 10 first** - These appear in almost every interview
- 2. Top 25 covers 80% of interview questions** - Focus heavily on these
- 3. Top 50 includes design principles (SOLID)** - Essential for senior roles
- 4. Top 100 for complete mastery** - Especially important for language-specific interviews (C++, Java, Python)

Key Areas to Practice:

- Code examples** for each concept (not just theory)

- **Real-world analogies** to explain complex concepts
- **Comparison tables** (differences between concepts)
- **Edge cases** (diamond problem, virtual destructors, etc.)
- **Design patterns** that use these principles

Common Interview Scenarios:

- **Fresher roles:** Top 25 questions
- **Mid-level (SDE 1-2):** Top 50 questions
- **Senior roles:** All 100 questions + system design using OOP